

Menu

CHƯƠNG 1: TỔNG QUAN THÔNG TIN.....	5
1.1. Tiêu chuẩn dự án phần mềm được lựa chọn.....	5
1.2. Thông tin nguồn gốc tài liệu.....	6
1.3. Thông tin về bên đánh giá.....	6
1.4. Mục đích và mục tiêu đánh giá.....	6
CHƯƠNG 2: GIỚI THIỆU THÔNG TIN DỰ ÁN ĐƯỢC ĐÁNH GIÁ CHẤT LƯỢNG.....	7
2.1. Giới thiệu dự án.....	7
2.2. Kế hoạch thực hiện.....	7
2.3. Mục tiêu.....	10
2.4. Đối tượng sử dụng.....	11
2.5. Phạm vi.....	11
2.6. Công nghệ phát triển.....	12
2.7. Sơ đồ phân rã chức năng.....	13
Chương 3: Test Plan.....	14
3.1. Giới thiệu Test Plan.....	14
3.2. Mục tiêu kiểm thử (Test Objectives).....	14
3.3. Phạm vi kiểm thử (Testing Scope).....	15
3.3.1. Các chức năng trong phạm vi kiểm thử (In Scope).....	15
3.3.2. Các chức năng ngoài phạm vi kiểm thử (Out of Scope).....	16
3.4. Chiến lược kiểm thử (Test Strategy).....	16
3.4.1. Unit Testing.....	16
3.4.2. Integration Testing.....	17
3.4.3. Manual Functional Testing.....	18
3.4.4. Automation Testing.....	19
3.4.5. Performance Testing.....	20
3.5. Môi trường kiểm thử (Test Environment).....	21
3.5.1. Hardware Environment.....	21
3.5.2. Software Environment.....	21
3.5.3. Application Environment.....	21
3.6. Công cụ kiểm thử (Testing Tools).....	21
3.7. Tiêu chí bắt đầu kiểm thử (Entry Criteria).....	22

3.8. Tiêu chí kết thúc kiểm thử (Exit Criteria).....	23
3.9. Rủi ro trong quá trình kiểm thử (Testing Risks)	23
3.10. Kết luận.....	24
CHƯƠNG 4: XÂY DỰNG VÀ THỰC HIỆN TEST CASE.....	25
4.1. Tổng quan về Test Case.....	25
4.2. Manual Test Case	26
4.2.1. Login Testing.....	26
4.2.2. Admin Testing	27
4.2.3. Order and Cart Testing	31
4.2.4. Kitchen Testing	33
4.2.5. Payment and Table Testing.....	35
4.3. Unit Testing	37
4.3.1. Phạm vi kiểm thử	37
4.3.2. Công cụ và phương pháp	38
4.3.3. Kết quả Coverage của Backend	38
4.3.4. Kết quả theo package	38
4.3.5. Đánh giá kết quả Unit Testing	39
4.4 Integration Testing.....	39
4.4.1. Phạm vi Integration Testing	40
4.4.2. Kiểm thử tích hợp giữa các tầng Backend	40
4.4.3. Kiểm thử tích hợp Frontend và Backend.....	40
4.4.4. Kiểm thử tích hợp Authentication và Authorization	41
4.4.5. Kiểm thử tích hợp Order và Kitchen	41
4.4.6. Kiểm thử tích hợp Order, Payment và Table	42
4.4.7. Kiểm thử tích hợp với cơ sở dữ liệu	42
4.4.8. Kết quả Coverage liên quan đến Integration Testing	42
4.4.9. Đánh giá Integration Testing	43
4.5. Automation Testing	43
4.5.1. Mục tiêu và phạm vi Automation Testing	43
4.5.2. Công cụ và phương pháp thực hiện Automation Testing.....	44
4.5.3. Chi tiết các Test Case Automation	45
4.5.4. Kết quả thực thi Automation Testing.....	46
4.5.5. Đánh giá Automation Testing.....	47

4.5.6. Kết luận Automation Testing	48
4.6. Performance Testing	48
4.6.1. Mục tiêu kiểm thử hiệu năng.....	48
4.6.2. Môi trường và cấu hình kiểm thử.....	49
4.6.3. Kịch bản kiểm thử hiệu năng.....	49
4.6.4. Kết quả kiểm thử hiệu năng	50
4.6.5. Phân tích kết quả	51
4.6.6. Đánh giá Performance Testing	52
4.7. Defect Management	52
4.7.1. Tổng quan Defect Management	52
4.7.2. Các nhóm defect được phát hiện.....	53
4.7.3. Defect trong Admin Module	53
4.7.4. Defect trong Order and Cart.....	53
4.7.5. Defect trong Payment and Table.....	54
4.7.6. Đánh giá Defect	54
4.8. Tổng hợp và đánh giá kết quả kiểm thử	54
4.8.1. Tổng hợp kết quả kiểm thử	54
4.8.2. Đánh giá kết quả Functional Testing	55
4.8.3. Đánh giá Code Coverage	55
4.8.4. Đánh giá Automation Testing.....	56
4.8.5. Đánh giá Performance Testing	56
4.8.6. Các vấn đề còn tồn tại	57
4.8.7. Đánh giá tổng thể	57
4.9. Kết luận chương	58
Chương 5. Đánh giá và Kết luận.....	60
5.1 Đánh giá kết quả đạt được	60
5.2. Ưu điểm	60
5.2.1. Ưu điểm về quy trình kiểm thử.....	60
5.2.2. Ưu điểm về phạm vi kiểm thử	61
5.2.3. Ưu điểm về kiểm thử tự động.....	61
5.2.4. Ưu điểm về đánh giá chất lượng mã nguồn	61
5.2.5. Ưu điểm về kiểm thử hiệu năng.....	61
5.2.6. Ưu điểm về phát hiện và phân tích lỗi	61

5.2.7. Đánh giá chung	62
5.3. Hạn chế	62
5.3.1. Hạn chế về phạm vi kiểm thử	62
5.3.2. Hạn chế về Automation Testing.....	62
5.3.3. Hạn chế về Code Coverage	62
5.3.4. Hạn chế về Performance Testing	63
5.3.5. Hạn chế về môi trường kiểm thử.....	63
5.3.6. Hạn chế trong việc quản lý Defect	63
5.3.7. Đánh giá chung về hạn chế	63
5.4. Hướng phát triển.....	63
5.4.1. Mở rộng phạm vi Automation Testing.....	63
5.4.2. Cải thiện Code Coverage	64
5.4.3. Mở rộng Performance Testing	64
5.4.4. Mở rộng môi trường kiểm thử	64
5.4.5. Cải thiện quản lý Defect	65
5.4.6. Tích hợp Automation Testing vào CI/CD	65
5.4.7. Hướng phát triển tổng thể	65
5.5. Kết luận.....	65

CHƯƠNG 1: TỔNG QUAN THÔNG TIN

1.1. Tiêu chuẩn dự án phần mềm được lựa chọn

Tiêu chuẩn dự án phần mềm được lựa chọn để làm đầu vào cho môn học đảm bảo chất lượng phần mềm bao gồm các tiêu chí sau:

- Dự án phần mềm đã hoàn thành trải qua các giai đoạn: Xác định yêu cầu, phân tích, thiết kế, lập trình, kiểm thử.
- Dự án phần mềm cần có các sản phẩm sau:
 - + Kế hoạch dự án (kế hoạch tổng thể, kế hoạch chi tiết giai đoạn)
 - + Tài liệu phân tích yêu cầu người sử dụng
 - + Tài liệu đặc tả yêu cầu phần mềm
 - + Tài liệu thiết kế (thiết kế CSDL, thiết kế hàm thủ tục, thiết kế kiến trúc, thiết kế màn hình)
 - + Source code/ coding convention
 - + Tài liệu kiểm thử (Tài liệu kịch bản kiểm thử, Báo cáo kiểm thử)
 - + Tài liệu hướng dẫn sử dụng
 - + Tài liệu hướng dẫn cài đặt
 - + Tài liệu đào tạo
 - + Tài liệu triển khai
 - + Tài liệu hỗ trợ
 - + Biên bản họp nội bộ/ Biên bản họp với khách hàng
 - + Thông tin thực tế khi triển khai dự án: nhật ký triển khai, vấn đề dự án, lỗi phần mềm, rủi ro dự án...
- Dự án phần mềm đã có các sản phẩm sau:
 - + Kế hoạch dự án
 - + Tài liệu đặc tả yêu cầu phần mềm
 - + Tài liệu thiết kế (thiết kế cơ sở dữ liệu, thiết kế kiến trúc hệ thống, thiết kế giao diện màn hình)
 - + Source code

1.2. Thông tin nguồn gốc tài liệu

- Thông tin nguồn gốc tài liệu đạt tiêu chuẩn dự án phần mềm được lựa chọn là:
 - + Đề tài: Phát triển phần mềm quản lý bán hàng cửa hàng đồ ăn nhanh FAF
 - + Giảng viên hướng dẫn: ThS. Trương Hữu Thành
 - + Sinh viên thực hiện: Trần Khánh Duy

1.3. Thông tin về bên đánh giá

Nhóm sinh viên thực hiện : Lê Thiên Phú

Thời gian thực hiện : 19/07/2026 đến ngày 01/08/2026

1.4. Mục đích và mục tiêu đánh giá

- Mục đích đánh giá:
 - + Đảm bảo chất lượng hệ thống phần mềm thông qua việc rà soát toàn diện dự án, nhằm phát hiện và khắc phục các khiếm khuyết.
 - + Đúc rút những bài học kinh nghiệm thực tiễn về công tác đảm bảo chất lượng trong suốt quá trình phát triển phần mềm.
 - + Xây dựng và đề xuất các phương án, kế hoạch kiểm soát chất lượng hiệu quả cho những dự án triển khai tiếp theo.
- Mục tiêu đánh giá:
 - + Hoàn thành triệt để tiến trình rà soát cho từng giai đoạn cụ thể trong vòng đời phát triển của dự án.
 - + Vận dụng hiệu quả các kiến thức lý thuyết đã học về "Đảm bảo chất lượng phần mềm" vào một dự án thực tiễn, qua đó góp phần nâng cao chất lượng đầu ra của sản phẩm.
 - + Nhận diện và chỉ ra chính xác các lỗi hoặc thiếu sót còn tồn tại trong quá trình xây dựng dự án.
 - + Giúp tất cả các thành viên tham gia tích lũy thêm kiến thức và kinh nghiệm thực tiễn về quy trình kiểm soát, đảm bảo chất lượng khi sản xuất phần mềm.

CHƯƠNG 2: GIỚI THIỆU THÔNG TIN DỰ ÁN ĐƯỢC ĐÁNH GIÁ CHẤT LƯỢNG

2.1. Giới thiệu dự án

- Tên đề tài: Phát triển phần mềm quản lý bán hàng cửa hàng đồ ăn nhanh FAF
- Nhiệm vụ đề tài:
 - + Thực hiện phân tích sâu quy trình vận hành tại các chuỗi cửa hàng đồ ăn nhanh hiện nay. Tập trung nhận diện các điểm nghẽn trong quản lý sản phẩm, luồng đơn hàng và quy trình thanh toán nhằm xây dựng giải pháp sát với thực tế kinh doanh.
 - + Xây dựng tài liệu phân tích chi tiết, xác định rõ ràng các yêu cầu chức năng và yêu cầu phi chức năng. Đảm bảo hệ thống có khả năng mở rộng, tính bảo mật cao và giao diện tối ưu cho người dùng.
 - + Xây dựng và lập trình các module chức năng của hệ thống như quản lý nhà bếp, quản lý kho, phục vụ khách hàng, và quản lý giao dịch, đảm bảo đáp ứng quy trình nghiệp vụ của cửa hàng.
 - + Thực hiện quy trình kiểm thử nghiêm ngặt (Unit Test, Integration Test và UAT). Đảm bảo hệ thống vận hành ổn định, không lỗi phát sinh trước khi bàn giao và đưa vào môi trường vận hành thực tế tại cửa hàng.

2.2. Kế hoạch thực hiện

Giai đoạn	STT	Công việc	Kết quả như thế nào là đạt	Ngày bắt đầu	Ngày kết thúc
Giai đoạn 1: Khảo sát	1	Khảo sát hoạt động bán hàng của cửa hàng đồ ăn nhanh: quy trình phục vụ khách, quy trình chế biến, thanh toán và quản lý kho	Hoàn thành tài liệu khảo sát gồm: mô tả nghiệp vụ phục vụ khách hàng, nhà bếp, giao dịch và kho	10/02/2006	13/02/2026
	2	Xác định yêu cầu chức năng và phi chức năng của hệ thống	Danh sách yêu cầu hệ thống gồm: quản lý phục vụ khách, quản lý bếp, giao dịch và kho	14/02/2026	16/02/2026

	3	Đánh giá lần 1 các công việc đã thực hiện	Họp nhóm rà soát tài liệu khảo sát và yêu cầu hệ thống, lập biên bản họp	16/02/2026	16/02/2026
Giai đoạn 2: Phân tích	4	Xây dựng mô hình nghiệp vụ và sơ đồ hệ thống gồm: sơ đồ BFD, DFD, mô tả chức năng hệ thống	Hoàn thành các mô hình: BFD quản lý bán hàng cửa hàng đồ ăn nhanh gồm 4 nhóm chức năng: phục vụ khách hàng, quản lý nhà bếp, quản lý giao dịch, quản lý kho	17/02/2026	21/02/2026
	5	Đánh giá lần 2 các mô hình phân tích	Nhóm họp rà soát mô hình nghiệp vụ và sơ đồ hệ thống, lập biên bản đánh giá	22/02/2026	23/02/2026
Giai đoạn 3: Thiết kế	6	Thiết kế hệ thống gồm: thiết kế giao diện đặt món, thiết kế CSDL quản lý đơn hàng – nguyên liệu – kho, thiết kế xử lý nghiệp vụ	Hoàn thành bản thiết kế giao diện hệ thống, thiết kế cơ sở dữ liệu đạt chuẩn, thiết kế kiểm soát dữ liệu nhập xuất	24/02/2026	01/03/2026
	7	Đánh giá lần 3 các tài liệu thiết kế	Họp nhóm rà soát tài liệu thiết kế, đảm bảo đáp ứng các yêu cầu chức năng và phi chức năng	02/03/2026	02/03/2026
Giai đoạn 4: Phát triển phần mềm	8	Viết test case và test data cho các module theo BFD	Hoàn thành danh sách testcase cho các chức năng: phục vụ khách, chế biến, giao dịch và kho	03/03/2026	05/03/2026

	9	Lập trình hệ thống quản lý bán hàng cửa hàng đồ ăn nhanh	Hoàn thành source code theo kiến trúc hệ thống, đáp ứng các chức năng trong BFD	06/03/2026	06/04/2026
	10	Đánh giá lần 4 tiến độ phát triển	Họp nhóm rà soát tiến độ lập trình và test case, lập biên bản họp	07/04/2026	07/04/2026
Giai đoạn 5: Kiểm thử	11	Rà soát chất lượng sản phẩm giữa các pha	Xác định danh sách sản phẩm chuyển giao giữa các pha và lập biên bản rà soát	08/04/2026	09/04/2026
	12	Kiểm thử các chức năng của hệ thống	Thực hiện test theo testcase cho các chức năng: phục vụ khách hàng, nhà bếp, giao dịch, kho	10/04/2026	13/04/2026
	13	Đánh giá lần 5 kết quả kiểm thử	Họp nhóm đánh giá kết quả test, đề xuất sửa lỗi và hoàn thiện hệ thống	14/04/2026	14/04/2026
Giai đoạn 6: Triển khai hệ thống	14	Hoàn thiện tài liệu hướng dẫn sử dụng hệ thống	Tài liệu hướng dẫn sử dụng cho nhân viên phục vụ, bếp, thu ngân và quản lý	15/04/2026	20/04/2026

	15	Đánh giá cuối cùng dự án	Tổng kết dự án, nghiệm thu hệ thống và lập biên bản đánh giá cuối cùng	21/04/2026	25/04/2026
--	----	--------------------------	--	------------	------------

2.3. Mục tiêu

- Xây dựng và triển khai hệ thống quản trị vận hành toàn diện cho cửa hàng đồ ăn nhanh, tập trung vào việc tối ưu hóa quy trình từ khâu tiếp nhận order, chế biến, thanh toán đến quản lý tồn kho vật tư. Hệ thống hướng tới việc số hóa quy trình, giảm thiểu sai sót thủ công và nâng cao trải nghiệm khách hàng.
- Lưu trữ và quản lý thông tin đơn đặt món, hóa đơn, nguyên liệu và các hoạt động của cửa hàng một cách chính xác, đầy đủ và dễ dàng tra cứu.
- Xây dựng hệ thống có đầy đủ các tính năng cần thiết như: hiển thị menu điện tử cho khách hàng, đặt món trực tiếp tại bàn, quản lý chế biến món ăn tại bếp, lập hóa đơn thanh toán và quản lý nhập – xuất kho nguyên liệu.
- Đáp ứng nhu cầu quản lý của cửa hàng đồ ăn nhanh, hỗ trợ các bộ phận phục vụ khách hàng, nhà bếp, thu ngân và quản lý kho thực hiện công việc nhanh chóng và hiệu quả.
- Giảm thiểu các thao tác thủ công trong quá trình phục vụ và quản lý, giúp tiết kiệm thời gian, công sức cho nhân viên và nâng cao hiệu quả hoạt động của cửa hàng.
- Đảm bảo phân quyền rõ ràng giữa các bộ phận trong hệ thống như: nhân viên phục vụ, nhân viên bếp, thu ngân và quản lý kho, giúp mỗi người chỉ truy cập và sử dụng các chức năng phù hợp với vai trò của mình.
- Hỗ trợ việc tổng hợp dữ liệu bán hàng, thống kê, kiểm kê nguyên liệu và lập báo cáo để phục vụ công tác quản lý và ra quyết định.
- Hệ thống có khả năng tích hợp và tương thích với các thiết bị và phần mềm liên quan trong cửa hàng như máy tính bảng đặt tại bàn, máy tính tại quầy thu ngân và khu vực bếp.
- Dự án cam kết hoàn thành đúng tiến độ đã đề ra, đảm bảo các tiêu chuẩn về an toàn dữ liệu và tính khả thi khi đưa vào vận hành thực tế tại doanh nghiệp.

Đảm bảo:

- 04 quy trình nghiệp vụ chính:
 - + Quy trình phục vụ khách hàng và đặt món
 - + Quy trình chế biến và phục vụ món ăn
 - + Quy trình quản lý giao dịch và thanh toán
 - + Quy trình quản lý kho và nguyên liệu

Các yêu cầu phi chức năng:

- Tính bảo mật:
 - + Hệ thống phải đảm bảo phân quyền rõ ràng cho từng bộ phận trong cửa hàng như nhân viên phục vụ, nhân viên bếp, thu ngân và quản lý kho.

- + Thông tin đơn hàng, hóa đơn thanh toán và dữ liệu nguyên liệu trong kho phải được bảo vệ khỏi truy cập trái phép.
- + Tài khoản và mật khẩu của nhân viên phải được quản lý an toàn, đảm bảo chỉ những người có quyền mới có thể truy cập và sử dụng các chức năng quản lý của hệ thống.
- Tính hữu dụng:
 - + Hệ thống phải cung cấp giao diện dễ sử dụng cho cả khách hàng và nhân viên.
 - + Menu điện tử trên máy tính bảng giúp khách hàng dễ dàng xem và đặt món.
 - + Các chức năng quản lý trên hệ thống như lập hóa đơn, quản lý kho và cập nhật trạng thái món ăn phải dễ thao tác và dễ cập nhật thông tin.
 - + Hệ thống phải hỗ trợ xuất các báo cáo bán hàng, báo cáo nguyên liệu và các thông tin cần thiết cho việc quản lý cửa hàng.
- Độ tin cậy:
 - + Mỗi nhân viên trong hệ thống phải có tài khoản riêng để truy cập hệ thống.
 - + Các thông tin về đơn đặt món, hóa đơn thanh toán và dữ liệu kho phải được lưu trữ chính xác và cập nhật kịp thời trong hệ thống.
 - + Hệ thống phải đảm bảo hoạt động ổn định trong quá trình phục vụ khách hàng tại cửa hàng.
- Tính hiệu năng:
 - + Hệ thống phải xử lý các thao tác đặt món, cập nhật trạng thái món và thanh toán một cách nhanh chóng.
 - + Thời gian phản hồi trung bình của hệ thống phải nhỏ hơn 2 giây khi khách hàng đặt món hoặc nhân viên thao tác trên hệ thống.
 - + Hệ thống phải cho phép nhiều bàn đặt món đồng thời mà vẫn đảm bảo hoạt động ổn định.
- Tính khả chuyển:
 - + Hệ thống phải được thiết kế để có thể triển khai trên các thiết bị máy tính và máy tính bảng trong cửa hàng.
 - + Dữ liệu của hệ thống phải có khả năng sao lưu và chuyển đổi sang hệ thống khác khi cần thiết mà vẫn đảm bảo hoạt động bình thường.

2.4. Đối tượng sử dụng

- Hệ thống được sử dụng bởi các bộ phận trong cửa hàng đồ ăn nhanh, bao gồm:
 - + Khách hàng: sử dụng máy tính bảng tại bàn để xem menu, đặt món và gửi phản hồi về món ăn.
 - + Nhân viên bếp: tiếp nhận đơn đặt món từ hệ thống, thực hiện chế biến và cập nhật trạng thái món ăn.
 - + Nhân viên thu ngân: quản lý hóa đơn, thực hiện thanh toán và xử lý các giao dịch bán hàng.
 - + Quản lý cửa hàng: theo dõi hoạt động bán hàng, quản lý hệ thống và xem các báo cáo liên quan.

2.5. Phạm vi

- Tên phần mềm: Hệ thống quản lý bán hàng cửa hàng đồ ăn nhanh FAF.
- Phần mềm được xây dựng nhằm hỗ trợ quản lý các hoạt động chính của cửa hàng đồ ăn nhanh như:

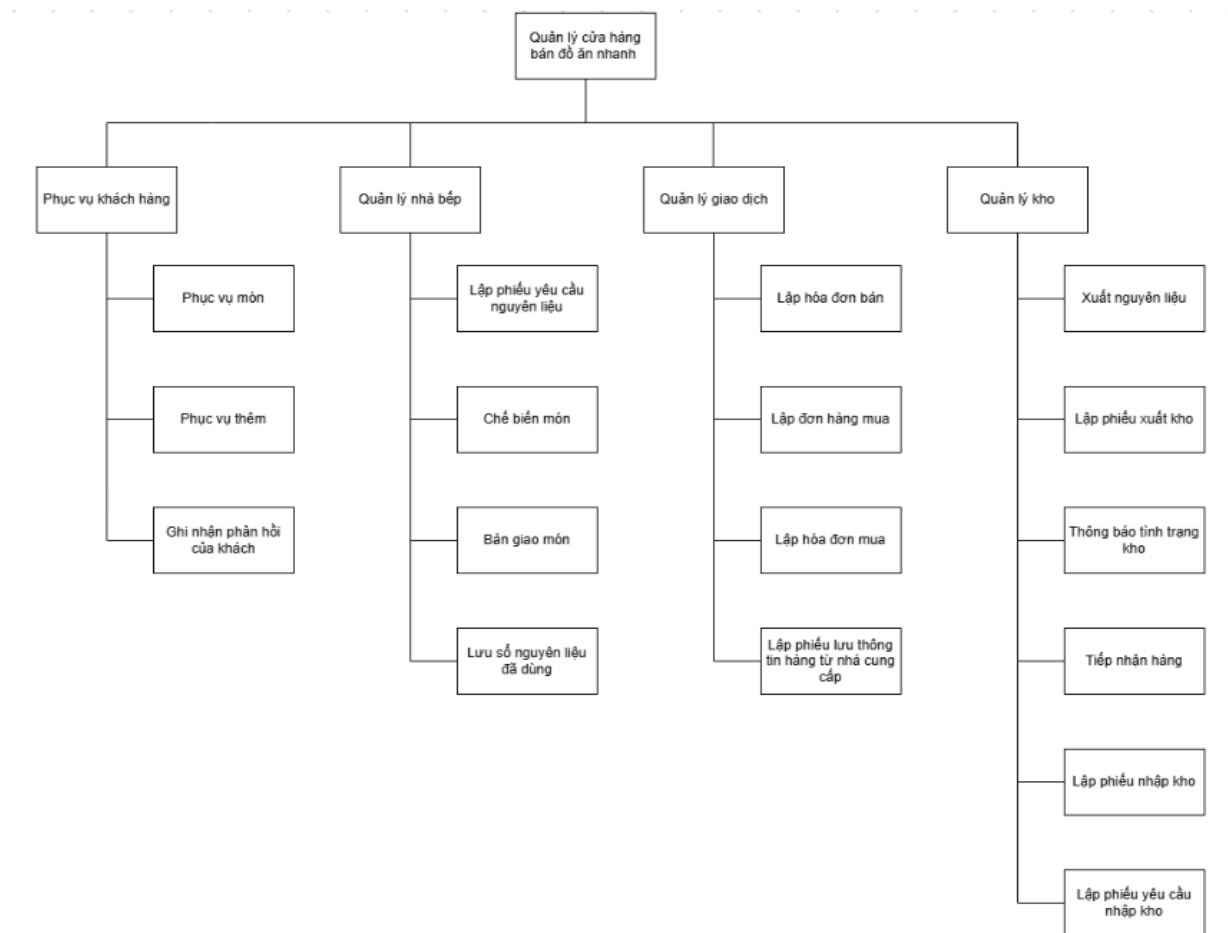
- + Quản lý menu và đặt món tại bàn thông qua máy tính bảng.
- + Quản lý quá trình chế biến món ăn tại khu vực bếp.
- + Quản lý giao dịch bán hàng và thanh toán hóa đơn.
- + Quản lý kho nguyên liệu, nhập kho, xuất kho và kiểm kê nguyên liệu.
- Hệ thống giúp các bộ phận trong cửa hàng phối hợp làm việc hiệu quả hơn, giảm thiểu các thao tác thủ công và nâng cao tốc độ phục vụ khách hàng.

2.6. Công nghệ phát triển

Hệ thống Quản lý bán hàng FAF được thiết kế phân lớp theo kiến trúc Client – Server. Sự phân tách rạch ròi giữa giao diện người dùng và máy chủ xử lý dữ liệu giúp hệ thống dễ dàng nâng cấp, đáp ứng tốt luồng nghiệp vụ phức tạp của cửa hàng. Hệ sinh thái công nghệ được nhóm lựa chọn bao gồm:

- Kiến trúc Backend (Máy chủ xử lý nghiệp vụ):
 - + Công nghệ sử dụng: Java Spring Boot.
 - + Mục đích & Đánh giá: Spring Boot cung cấp một bộ khung (framework) vững chắc để xây dựng các API RESTful phục vụ cho mọi chức năng của hệ thống. Việc ứng dụng công nghệ này giúp hệ thống đảm bảo được tính bảo mật, xử lý dữ liệu hiệu quả, đồng thời mang lại khả năng mở rộng (scalability) và dễ dàng bảo trì trong tương lai.
- Kiến trúc Frontend (Giao diện người dùng):
 - + Công nghệ sử dụng: ReactJS.
 - + Mục đích & Đánh giá: Nhằm mục tiêu tối ưu trải nghiệm người dùng, ReactJS được lựa chọn để xây dựng một giao diện hiện đại, dễ sử dụng và có khả năng tương tác cao. Khả năng render mượt mà của ReactJS đặc biệt phù hợp với các thao tác nhanh, liên tục của nhân viên phục vụ, đầu bếp hay thu ngân tại cửa hàng.
- Hệ quản trị Cơ sở dữ liệu (Database):
 - + Công nghệ sử dụng: MySQL.
 - + Mục đích & Đánh giá: Dữ liệu kinh doanh cần tính toàn vẹn và độ tin cậy tuyệt đối. Do đó, MySQL - một hệ quản trị cơ sở dữ liệu quan hệ mạnh mẽ - được dùng để lưu trữ toàn bộ các thông tin cốt lõi như: danh mục menu, đơn hàng, hóa đơn, nguyên liệu kho và các thông tin liên quan khác của hệ thống.
- Quản lý mã nguồn và Quy trình làm việc (Version Control):
 - + Công cụ sử dụng: Git.
 - + Mục đích & Đánh giá: Để đảm bảo quy trình làm việc nhóm chuyên nghiệp, dự án sử dụng Git làm hệ thống quản lý phiên bản mã nguồn. Công cụ này cho phép các thành viên lập trình đồng bộ, dễ dàng theo dõi, truy xuất các thay đổi và hỗ trợ cực tốt cho công tác xử lý xung đột (conflict) trong quá trình phát triển và kiểm thử hệ thống.

2.7. Sơ đồ phân rã chức năng



Chương 3: Test Plan

3.1. Giới thiệu Test Plan

Test Plan là tài liệu xác định định hướng và kế hoạch kiểm thử cho hệ thống FastFood Management System. Tài liệu này xác định mục tiêu, phạm vi, phương pháp, môi trường, công cụ, tiêu chí và các rủi ro liên quan đến quá trình kiểm thử hệ thống.

Hệ thống FastFood Management System là hệ thống quản lý bán hàng cho cửa hàng đồ ăn nhanh, được phát triển theo mô hình Full-stack, bao gồm:

- Frontend: ReactJS + Vite.
- Backend: Java Spring Boot REST API.
- Database: MySQL.
- Unit Testing: JUnit 5, Mockito.
- Integration Testing: Spring Boot Test.
- Automation Testing: Selenium WebDriver, TestNG.
- Performance Testing: Apache JMeter.

Hệ thống hỗ trợ các nghiệp vụ chính như quản lý tài khoản và phân quyền, quản lý món ăn, quản lý nguyên liệu, quản lý bàn, quản lý đơn hàng, thanh toán và theo dõi trạng thái chế biến món ăn. Kế hoạch kiểm thử được xây dựng nhằm định hướng quá trình đánh giá chất lượng hệ thống thông qua kiểm thử chức năng thủ công, kiểm thử đơn vị, kiểm thử tích hợp, kiểm thử tự động giao diện và kiểm thử hiệu năng. Kết quả kiểm thử được sử dụng để phát hiện lỗi, đánh giá mức độ đáp ứng của hệ thống và đề xuất các biện pháp cải thiện chất lượng phần mềm.

3.2. Mục tiêu kiểm thử (Test Objectives)

Mục tiêu của quá trình kiểm thử là đánh giá mức độ chính xác, ổn định và khả năng đáp ứng yêu cầu của hệ thống FastFood Management System.

Các mục tiêu cụ thể bao gồm:

Kiểm tra các chức năng của hệ thống có hoạt động đúng theo yêu cầu và quy trình nghiệp vụ hay không.

Kiểm tra tính hợp lệ của dữ liệu đầu vào và khả năng xử lý dữ liệu không hợp lệ của hệ thống.

Kiểm tra chức năng đăng nhập và phân quyền, đảm bảo người dùng chỉ có thể truy cập các chức năng phù hợp với vai trò.

Kiểm tra các nghiệp vụ chính như quản lý món ăn, đặt món, quản lý đơn hàng, xử lý món ăn tại bếp và thanh toán.

Kiểm tra sự tương tác giữa các thành phần Frontend, Backend và Database.

Kiểm tra các phương thức và thành phần xử lý nghiệp vụ thông qua Unit Testing.

Kiểm tra sự phối hợp giữa các tầng Controller, Service, Repository và Database thông qua Integration Testing.

Tự động hóa một số luồng nghiệp vụ quan trọng bằng Selenium WebDriver và TestNG nhằm kiểm tra khả năng hoạt động của hệ thống trên giao diện người dùng.

Đánh giá khả năng xử lý của hệ thống khi có nhiều yêu cầu được gửi đồng thời bằng Apache JMeter.

Phát hiện, ghi nhận và phân tích các lỗi phát sinh trong quá trình kiểm thử.

Cung cấp kết quả kiểm thử làm cơ sở để đánh giá chất lượng và đề xuất cải tiến hệ thống.

3.3. Phạm vi kiểm thử (Testing Scope)

Phạm vi kiểm thử được xác định dựa trên các chức năng chính của hệ thống và các phương pháp kiểm thử được áp dụng trong quá trình đánh giá.

3.3.1. Các chức năng trong phạm vi kiểm thử (In Scope)

Các chức năng được đưa vào phạm vi kiểm thử bao gồm:

1. Authentication và Authorization

- Đăng nhập hệ thống.
- Kiểm tra username và password.
- Kiểm tra dữ liệu đầu vào khi đăng nhập.
- Kiểm tra thông báo khi đăng nhập không thành công.
- Kiểm tra quyền truy cập theo vai trò người dùng.

Các vai trò được xem xét trong quá trình kiểm thử gồm:

Vai trò	Phạm vi chức năng
ADMIN	Quản lý và theo dõi các chức năng quản trị hệ thống
Thu ngân	Quản lý đơn hàng và thực hiện thanh toán
Bếp	Tiếp nhận và xử lý món ăn
Bàn	Xem menu và thực hiện đặt món

2. Food Management

- Hiển thị danh sách món ăn.
- Thêm món ăn.
- Cập nhật thông tin món ăn.
- Xóa món ăn.
- Kiểm tra dữ liệu món ăn.

3. Order Management

- Xem và lựa chọn món ăn.
- Thêm món vào giỏ hàng.
- Tạo đơn hàng.
- Kiểm tra thông tin đơn hàng.
- Cập nhật trạng thái đơn hàng.

4. Kitchen Management

- Tiếp nhận đơn hàng.
- Hiển thị các món cần chế biến.
- Cập nhật trạng thái chế biến món ăn.

- Kiểm tra trạng thái món sau khi cập nhật.

5. Payment Management

- Hiển thị thông tin hóa đơn.
- Kiểm tra tổng tiền.
- Thực hiện thanh toán.
- Kiểm tra trạng thái đơn hàng sau thanh toán.

6. Inventory Management

- Quản lý nguyên liệu.
- Nhập kho.
- Cập nhật số lượng tồn kho.
- Kiểm tra việc thay đổi số lượng nguyên liệu khi phát sinh nghiệp vụ liên quan.

7. Table Management

- Hiển thị danh sách bàn.
- Kiểm tra trạng thái bàn.
- Cập nhật trạng thái bàn khi sử dụng.
- Kiểm tra trạng thái bàn sau khi hoàn tất thanh toán.

8. Performance Testing

- Kiểm tra thời gian phản hồi của hệ thống.
- Kiểm tra khả năng xử lý nhiều request đồng thời.
- Kiểm tra tỷ lệ request thành công và thất bại.
- Phân tích các lỗi phát sinh trong điều kiện nhiều người dùng đồng thời.

3.3.2. Các chức năng ngoài phạm vi kiểm thử (Out of Scope)

Các nội dung sau không thuộc phạm vi kiểm thử của báo cáo:

- Kiểm thử phần cứng máy chủ chuyên sâu.
- Kiểm thử hệ thống thanh toán với ngân hàng hoặc cổng thanh toán thực tế.
- Penetration Testing và Security Testing chuyên sâu.
- Kiểm thử khả năng chịu lỗi của hệ điều hành.
- Kiểm thử trên các thiết bị phần cứng chuyên dụng của cửa hàng.
- Stress Testing và Endurance Testing chuyên sâu nếu không nằm trong kịch bản kiểm thử được thực hiện.

3.4. Chiến lược kiểm thử (Test Strategy)

Hệ thống được kiểm thử theo nhiều mức độ khác nhau nhằm đảm bảo chất lượng phần mềm.

3.4.1. Unit Testing

Unit Testing được thực hiện do ban đầu nhóm phát triển không thực hiện. Unit Testing nhằm kiểm tra các thành phần nhỏ và độc lập của hệ thống, tập trung vào việc xác nhận các phương thức xử lý nghiệp vụ hoạt động đúng với các điều kiện đầu vào khác nhau.

Unit Testing được áp dụng chủ yếu cho các thành phần Controller, Service và Mapper của Backend.

Mục tiêu:

- Kiểm tra logic xử lý của từng thành phần.
- Kiểm tra dữ liệu đầu vào và đầu ra của phương thức.
- Kiểm tra các trường hợp dữ liệu hợp lệ và không hợp lệ.
- Kiểm tra việc xử lý exception.
- Phát hiện lỗi ở cấp độ thành phần trước khi thực hiện các mức kiểm thử cao hơn.

Công cụ sử dụng:

- JUnit 5.
- Mockito.

Phạm vi kiểm thử:

- Controller.
- Service.
- Mapper.
- Các phương thức xử lý nghiệp vụ liên quan.

Kết quả thực hiện Unit Testing được tổng hợp và trình bày tại Chương 4.

3.4.2. Integration Testing

Integration Testing được thực hiện nhằm kiểm tra sự tương tác và phối hợp giữa các thành phần của hệ thống, đặc biệt là luồng xử lý từ Controller đến Service, Repository và Database.

Mục tiêu:

- Kiểm tra sự kết nối giữa các tầng trong hệ thống.
- Kiểm tra việc xử lý request và response của API.
- Kiểm tra dữ liệu được truyền chính xác giữa các thành phần.
- Kiểm tra việc lưu trữ và truy xuất dữ liệu từ Database.
- Phát hiện lỗi phát sinh khi các thành phần hoạt động cùng nhau.

Các thành phần được kiểm thử:

- Controller.
- Service.
- Repository.
- Database.

Công cụ sử dụng:

- Spring Boot Test.
- MockMvc.
- JPA Repository.

Một Integration Test được đánh giá dựa trên kết quả của toàn bộ luồng xử lý, bao gồm request đầu vào, response trả về và dữ liệu tương ứng trong Database.

Kết quả thực hiện Integration Testing được trình bày tại Chương 4.

3.4.3. Manual Functional Testing

Manual Functional Testing được thực hiện nhằm kiểm tra các chức năng của hệ thống có hoạt động đúng theo yêu cầu nghiệp vụ hay không. Tester thực hiện trực tiếp các thao tác trên giao diện hệ thống và đối chiếu kết quả thực tế với kết quả mong đợi được xác định trong Test Case.

Mục tiêu:

- Kiểm tra tính đúng đắn của các chức năng.
- Kiểm tra dữ liệu đầu vào và kết quả đầu ra.
- Kiểm tra validation và thông báo lỗi.
- Kiểm tra các quy tắc nghiệp vụ.
- Kiểm tra quyền truy cập theo từng vai trò người dùng.
- Phát hiện các lỗi xảy ra trong quá trình sử dụng hệ thống.

Phương pháp thực hiện:

1. Phân tích yêu cầu và xác định chức năng cần kiểm thử.
2. Xây dựng Test Case và Test Data.
3. Chuẩn bị môi trường kiểm thử.
4. Thực hiện Test Case trên hệ thống.
5. Ghi nhận Actual Result.
6. So sánh Actual Result với Expected Result.
7. Xác định trạng thái PASS, FAIL hoặc BLOCKED.
8. Ghi nhận Defect đối với Test Case FAIL.
9. Thực hiện Retest sau khi Defect được sửa.
10. Thực hiện Regression Test đối với các chức năng có liên quan.

Các kỹ thuật được sử dụng:

- Positive Testing.
- Negative Testing.
- Boundary Value Testing.
- Validation Testing.
- Role-based Testing.

Các nhóm chức năng được kiểm thử thủ công:

- Login.
- Admin Management.
- Kitchen Management.
- Payment Management.
- Các chức năng nghiệp vụ liên quan nằm trong phạm vi kiểm thử.

Các Test Case thủ công được quản lý dưới dạng bảng Test Case, bao gồm Test Case ID, Test Scenario, Preconditions, Test Data, Test Steps, Expected Result, Actual Result và Status.

Chi tiết các Test Case và kết quả thực thi được trình bày tại Chương 4.

3.4.4. Automation Testing

Automation Testing được thực hiện nhằm tự động hóa một số kịch bản kiểm thử chức năng trên giao diện người dùng, giảm thời gian thực thi các Test Case lặp lại và hỗ trợ kiểm tra tính ổn định của các luồng nghiệp vụ quan trọng.

Trong phạm vi dự án, kiểm thử tự động được thực hiện trên giao diện Web bằng Selenium WebDriver kết hợp với TestNG. Các kịch bản được xây dựng theo mô hình Page Object nhằm tách biệt phần xử lý giao diện với phần thực thi Test Case, giúp mã nguồn kiểm thử dễ quản lý và bảo trì.

Mục tiêu:

- Tự động hóa các Test Case chức năng có tính lặp lại.
- Kiểm tra các luồng nghiệp vụ quan trọng trên giao diện thực tế.
- Kiểm tra sự tương tác giữa Frontend và Backend thông qua thao tác của người dùng.
- Giảm thời gian thực thi kiểm thử thủ công đối với các Test Case đã được tự động hóa.
- Hỗ trợ thực hiện Regression Testing sau khi hệ thống có thay đổi.

Công cụ sử dụng:

- Selenium WebDriver: điều khiển trình duyệt và mô phỏng thao tác người dùng.
- TestNG: tổ chức và thực thi các Test Case tự động.
- Maven: quản lý dependency và thực thi quá trình kiểm thử.
- Java: ngôn ngữ lập trình cho bộ Automation Test.

Phương pháp thực hiện:

1. Xác định các Test Case phù hợp để tự động hóa.
2. Xây dựng Page Object cho các trang và thành phần giao diện cần kiểm thử.
3. Xây dựng Test Data cho các trường hợp kiểm thử.
4. Viết Automation Test bằng Selenium WebDriver và TestNG.
5. Khởi động môi trường kiểm thử gồm Frontend, Backend và Database.
6. Thực thi Test Case tự động.
7. Kiểm tra kết quả thực tế dựa trên các điều kiện xác nhận được xây dựng trong Test Case.
8. Ghi nhận kết quả Pass/Fail.
9. Phân tích và xử lý lỗi phát sinh trong quá trình thực thi.

Các kịch bản Automation Test được thực hiện:

Test Case	Nội dung
Login Test	Kiểm tra đăng nhập bằng tài khoản hợp lệ và không hợp lệ
Add Cart Test	Kiểm tra thêm món ăn vào giỏ hàng
Order Test	Kiểm tra luồng tạo đơn hàng

Automation Testing tập trung vào các luồng nghiệp vụ quan trọng và có khả năng được thực thi lặp lại nhiều lần. Các Test Case tự động và kết quả thực thi được trình bày chi tiết tại Chương 4.

3.4.5. Performance Testing

Performance Testing được thực hiện nhằm đánh giá khả năng xử lý của hệ thống khi có nhiều người dùng hoặc nhiều yêu cầu được gửi đồng thời. Việc kiểm thử tập trung vào thời gian phản hồi, số lượng request được xử lý, tỷ lệ thành công và các lỗi phát sinh trong quá trình xử lý đồng thời.

Trong phạm vi dự án, Apache JMeter được sử dụng để mô phỏng nhiều người dùng gửi request đến Backend và theo dõi kết quả xử lý của hệ thống.

Mục tiêu:

- Đánh giá khả năng xử lý request của Backend.
- Đo lường thời gian phản hồi của hệ thống.
- Kiểm tra tỷ lệ request thành công và thất bại.
- Kiểm tra khả năng xử lý nhiều request đồng thời.
- Phát hiện các vấn đề liên quan đến xử lý đồng thời và tính toàn vẹn dữ liệu.

Công cụ sử dụng:

- Apache JMeter.
- Spring Boot Backend.
- MySQL Database.

Cấu hình kiểm thử:

Thông số	Giá trị
Concurrent Users	10
Loop Count	5
Tổng số Request	50
Server	Localhost
Backend	Spring Boot
Database	MySQL

Phương pháp thực hiện:

1. Xây dựng Test Plan trên Apache JMeter.
2. Cấu hình số lượng người dùng đồng thời.
3. Cấu hình số lần lặp của mỗi người dùng.
4. Gửi các request đến API của hệ thống.
5. Thu thập các thông số Response Time, Throughput và Error Rate.
6. Phân tích kết quả thực thi.
7. Xác định các lỗi phát sinh trong điều kiện xử lý đồng thời.

Trong quá trình kiểm thử, hệ thống xuất hiện lỗi Duplicate Key (Primary Key) trong một số trường hợp nhiều request đồng thời thực hiện thao tác tạo dữ liệu. Kết quả này cho thấy cơ chế sinh mã định danh hiện tại có vấn đề trong môi trường concurrent.

Nguyên nhân được xác định liên quan đến cơ chế sinh mã dựa trên việc lấy giá trị lớn nhất hiện có rồi tăng thêm một đơn vị. Khi nhiều transaction thực hiện đồng thời, các transaction có thể lấy cùng một giá trị và tạo ra cùng một mã định danh, dẫn đến xung đột khóa chính.

Vấn đề này được ghi nhận là một phát hiện của Performance Testing và được sử dụng làm cơ sở để đề xuất cải thiện cơ chế sinh khóa chính trong phần đánh giá chất lượng hệ thống.

3.5. Môi trường kiểm thử (Test Environment)

Môi trường kiểm thử được thiết lập nhằm đảm bảo các thành phần Frontend, Backend, Database và các công cụ kiểm thử có thể hoạt động đồng thời trong quá trình thực hiện Test Case.

3.5.1. Hardware Environment

Thành phần	Cấu hình
CPU	Intel Core i5
RAM	16 GB
Storage	SSD 1 TB
Operating System	Windows 11

3.5.2. Software Environment

Thành phần	Phiên bản/Công nghệ
Java	JDK 21
Backend	Spring Boot
Frontend	ReactJS + Vite
Database	MySQL 8.x
Build Tool	Maven 3.x
Browser	Google Chrome
Selenium	4.35.0
TestNG	7.11.0
JMeter	Apache JMeter 5.x

3.5.3. Application Environment

Hệ thống được triển khai trong môi trường kiểm thử cục bộ. Frontend và Backend được khởi động độc lập, kết nối với cơ sở dữ liệu MySQL phục vụ cho quá trình kiểm thử.

Đối với Automation Testing, Selenium WebDriver sử dụng trình duyệt Google Chrome để mô phỏng các thao tác của người dùng trên giao diện Web.

Đối với Performance Testing, Apache JMeter gửi request trực tiếp đến Backend nhằm đánh giá khả năng xử lý của hệ thống trong điều kiện nhiều request được thực hiện đồng thời.

3.6. Công cụ kiểm thử (Testing Tools)

Các công cụ được sử dụng trong quá trình phát triển và kiểm thử hệ thống được phân loại theo mục đích sử dụng như sau:

Công cụ	Mục đích sử dụng
IntelliJ IDEA	Phát triển và kiểm thử Backend
Visual Studio Code	Phát triển Frontend
JUnit 5	Thực hiện Unit Testing
Mockito	Mock các dependency trong Unit Testing
Spring Boot Test	Thực hiện Integration Testing
MockMvc	Kiểm thử các API Controller
Selenium WebDriver	Tự động hóa kiểm thử giao diện Web
TestNG	Tổ chức và thực thi Automation Test
Maven	Quản lý dependency và thực thi Test
Apache JMeter	Kiểm thử hiệu năng và khả năng xử lý đồng thời
MySQL	Lưu trữ dữ liệu phục vụ kiểm thử
JaCoCo	Đo độ bao phủ mã nguồn
SonarQube/SonarCloud	Phân tích chất lượng mã nguồn
Git/GitHub	Quản lý mã nguồn và phiên bản
GitHub Actions	Hỗ trợ xây dựng và thực thi quy trình CI/CD

Các công cụ được lựa chọn dựa trên từng mức kiểm thử và mục đích đánh giá. JUnit và Mockito được sử dụng ở mức Unit Testing; Spring Boot Test và MockMvc được sử dụng cho Integration Testing; Selenium và TestNG phục vụ Automation Testing; Apache JMeter được sử dụng cho Performance Testing.

3.7. Tiêu chí bắt đầu kiểm thử (Entry Criteria)

Quá trình kiểm thử được bắt đầu khi các điều kiện cần thiết cho việc thực hiện kiểm thử đã được đáp ứng.

Các tiêu chí bắt đầu kiểm thử bao gồm:

- Source code của hệ thống đã được build thành công.
- Frontend và Backend có thể khởi động bình thường.
- Database được tạo và có dữ liệu kiểm thử cần thiết.
- Các chức năng thuộc phạm vi kiểm thử đã được phát triển và sẵn sàng để kiểm thử.
- Test Case và Test Data cần thiết đã được chuẩn bị.
- Môi trường kiểm thử đã được cấu hình.
- Tài khoản kiểm thử cho các vai trò cần thiết đã được chuẩn bị.
- Các công cụ kiểm thử cần thiết đã được cài đặt và cấu hình.
- Đối với Automation Testing, trình duyệt và WebDriver có thể hoạt động bình thường.
- Đối với Performance Testing, API và môi trường Backend đã sẵn sàng nhận request kiểm thử.

3.8. Tiêu chí kết thúc kiểm thử (Exit Criteria)

Quá trình kiểm thử được xem là hoàn thành khi các điều kiện kết thúc kiểm thử được đáp ứng. Các tiêu chí kết thúc bao gồm:

- Các Test Case thuộc phạm vi kiểm thử đã được thực thi.
- Kết quả thực thi của từng Test Case đã được ghi nhận.
- Các Test Case có trạng thái FAIL đã được phân tích và ghi nhận Defect.
- Các lỗi nghiêm trọng được phát hiện trong quá trình kiểm thử đã được sửa hoặc được ghi nhận là vấn đề còn tồn tại.
- Các Defect sau khi sửa đã được thực hiện Retest.
- Các chức năng liên quan đến Defect đã được thực hiện Regression Test khi cần thiết.
- Các Automation Test thuộc phạm vi đã được thực thi và ghi nhận kết quả.
- Performance Test đã được thực hiện theo cấu hình được xác định trong Test Plan.
- Kết quả kiểm thử được tổng hợp và phân tích.
- Các vấn đề còn tồn tại và đề xuất cải thiện chất lượng được ghi nhận trong báo cáo.

3.9. Rủi ro trong quá trình kiểm thử (Testing Risks)

Trong quá trình kiểm thử hệ thống, một số rủi ro có thể ảnh hưởng đến kết quả kiểm thử được xác định như sau:

Rủi ro	Ảnh hưởng	Biện pháp xử lý
Database không đồng nhất với môi trường kiểm thử	Kết quả Test Case có thể không chính xác	Chuẩn bị Database và Test Data riêng cho môi trường kiểm thử
API thay đổi trong quá trình phát triển	Test Case và Automation Test có thể không còn phù hợp	Cập nhật Test Case và Automation Script khi API thay đổi
Selector của Selenium thay đổi	Automation Test có thể bị lỗi	Kiểm tra và cập nhật Locator/Selector
Môi trường kiểm thử không ổn định	Test Case có thể FAIL không phải do lỗi hệ thống	Kiểm tra Frontend, Backend và Database trước khi test
Thiếu Test Data	Một số Test Case không thể thực hiện	Chuẩn bị dữ liệu kiểm thử trước khi thực thi
Test thực thi chậm	Tăng thời gian kiểm thử	Tối ưu Test Case và Automation Script
Nhiều request đồng thời gây lỗi dữ liệu	Ảnh hưởng đến tính toàn vẹn dữ liệu	Phân tích log và kết quả JMeter, kiểm tra cơ chế xử lý transaction
Thay đổi chức năng sau khi Test Case được xây dựng	Test Case có thể không còn phù hợp	Thực hiện Review và cập nhật Test Case

3.10. Kết luận

Chương 3 đã trình bày kế hoạch kiểm thử cho hệ thống FastFood Management System, bao gồm mục tiêu, phạm vi, chiến lược, môi trường, công cụ, tiêu chí bắt đầu, tiêu chí kết thúc, lịch trình và các rủi ro trong quá trình kiểm thử.

Chiến lược kiểm thử kết hợp nhiều phương pháp gồm Manual Functional Testing, Unit Testing, Integration Testing, Automation Testing và Performance Testing nhằm đánh giá hệ thống ở nhiều mức độ khác nhau.

Các nội dung trong Test Plan là cơ sở để xây dựng Test Case, tổ chức thực hiện kiểm thử và tổng hợp kết quả kiểm thử được trình bày trong Chương 4.

CHƯƠNG 4: XÂY DỰNG VÀ THỰC HIỆN TEST CASE

4.1. Tổng quan về Test Case

Chương 4 trình bày quá trình thiết kế, thực hiện và tổng hợp kết quả kiểm thử đối với hệ thống FastFood Management System. Nội dung kiểm thử được xây dựng dựa trên các Test Case đã được chuẩn bị cho các nhóm chức năng chính của hệ thống.

Trong phạm vi Manual Functional Testing, bốn bộ Test Case được sử dụng để kiểm tra các chức năng Admin, Order/Cart, Kitchen và Payment/Table. Các Test Case bao gồm cả trường hợp chức năng hoạt động bình thường và các trường hợp dữ liệu không hợp lệ, lỗi hệ thống, lỗi kết nối, phiên đăng nhập hết hạn và các tình huống phát sinh trong quá trình xử lý nghiệp vụ.

Tổng cộng có 194 Test Case được thực hiện, bao gồm:

- 59 Test Case cho chức năng Admin.
- 18 Test Case cho chức năng Login
- 39 Test Case cho chức năng Order/Cart
- 38 Test Case cho chức năng Kitchen.
- 40 Test Case cho chức năng Payment/Table.

Kết quả thực thi được ghi nhận trực tiếp trong các file Test Case thông qua các trường Expected Result, Actual Result và Status. Kết quả tổng hợp gồm 165 Test Case Pass, 22 Test Case Fail, 6 Test Case Skipped, 1 Test Case Blocked.

Bên cạnh các trường hợp kiểm thử chức năng thông thường, bộ Test Case còn bao phủ nhiều tình huống bất thường và điều kiện biên. Ví dụ, nhóm Order/Cart kiểm tra các trường hợp giỏ hàng trống, mất kết nối mạng, server không phản hồi, phiên đăng nhập hết hạn và thay đổi số lượng sản phẩm liên tục. Nhóm Kitchen kiểm tra việc xử lý nhiều đơn hàng, đồng bộ trạng thái với Thu ngân và các trường hợp API cập nhật thất bại. Nhóm Payment/Table kiểm tra nhiều phương thức thanh toán, lỗi kết nối, thanh toán thất bại và vòng đời trạng thái của bàn. Nhóm Admin bao phủ Dashboard, quản lý món ăn, kho nguyên liệu, lịch sử nhập – tiêu thụ và một số trường hợp API lỗi.

Các kết quả kiểm thử được sử dụng để xác định những chức năng hoạt động đúng, những chức năng còn tồn tại lỗi và các trường hợp chưa thể hoặc chưa được thực hiện. Những Test Case có trạng thái Fail, Blocked, Skipped hoặc Not Run sẽ được phân tích ở các phần tiếp theo để xác định nguyên nhân và mức độ ảnh hưởng.

Chương này được tổ chức thành các nội dung chính gồm thiết kế và thực hiện Manual Functional Testing, kết quả kiểm thử theo từng nhóm chức năng, Unit Testing, Integration Testing, Automation Testing, Performance Testing, Code Coverage và tổng hợp các Defect được phát hiện trong quá trình đánh giá hệ thống.

4.2. Manual Test Case

4.2.1. Login Testing

Login Testing được thực hiện nhằm kiểm tra chức năng xác thực người dùng, khả năng xử lý dữ liệu đầu vào và cơ chế phân quyền của hệ thống. Bộ Test Case Login gồm 18 Test Case, được thực hiện với nhiều loại dữ liệu và tài khoản người dùng khác nhau.

Các Test Case được chia thành hai nhóm chính:

- Kiểm tra validation và xử lý dữ liệu đăng nhập.
- Kiểm tra đăng nhập thành công và chuyển hướng theo vai trò người dùng.

4.2.1.1. Kiểm tra validation và dữ liệu đầu vào

Các Test Case từ TC_LOGIN_001 đến TC_LOGIN_011 tập trung kiểm tra các trường hợp đăng nhập không hợp lệ và dữ liệu đầu vào bất thường.

Các trường hợp được kiểm tra bao gồm:

Test Case	Nội dung kiểm thử
TC_LOGIN_001	Đăng nhập với username không tồn tại
TC_LOGIN_002	Đăng nhập với password sai
TC_LOGIN_003	Bỏ trống username
TC_LOGIN_004	Bỏ trống password
TC_LOGIN_005	Bỏ trống cả username và password
TC_LOGIN_006	Kiểm tra username có khoảng trắng
TC_LOGIN_007	Kiểm tra password có khoảng trắng
TC_LOGIN_008	Kiểm tra username có độ dài lớn
TC_LOGIN_009	Kiểm tra dữ liệu chứa ký tự đặc biệt
TC_LOGIN_010	Kiểm tra SQL Injection payload
TC_LOGIN_011	Kiểm tra XSS payload

Các Test Case trên nhằm đảm bảo hệ thống không chỉ xử lý đúng thông tin đăng nhập hợp lệ mà còn có khả năng xử lý các dữ liệu đầu vào bất thường.

Kết quả thực thi cho thấy các trường hợp kiểm tra validation và dữ liệu không hợp lệ đều đạt trạng thái PASS. Hệ thống hiển thị validation hoặc thông báo lỗi phù hợp và không cho phép đăng nhập thành công với thông tin không hợp lệ.

4.2.1.2. Kiểm tra đăng nhập và phân quyền

Các Test Case từ TC_LOGIN_012 đến TC_LOGIN_018 kiểm tra khả năng đăng nhập bằng tài khoản hợp lệ và chuyển hướng người dùng đến giao diện tương ứng với vai trò.

Test Case	Role	Expected URL
TC_LOGIN_012	ADMIN	/admin/dashboard
TC_LOGIN_013	ThuNgan	/payment

TC_LOGIN_014	Bep	/kitchen
TC_LOGIN_015	Ban01	/menu
TC_LOGIN_016	Ban02	/menu
TC_LOGIN_017	Ban03	/menu
TC_LOGIN_018	Ban04	/menu

Kết quả kiểm thử cho thấy các tài khoản hợp lệ đều đăng nhập thành công và được chuyển hướng đến đúng giao diện tương ứng với vai trò được cấp.

Điều này cho thấy chức năng Authentication và Role-based Authorization trong phạm vi được kiểm thử hoạt động đúng với các tài khoản và vai trò được chuẩn bị.

4.2.1.3. Tổng hợp kết quả Login Testing

Tổng số Test Case	Pass	Fail	Skipped	Blocked	Not Run
18	18	0	0	0	0

Tỷ lệ Test Case Pass:

$$18 / 18 \times 100\% = 100\%$$

Như vậy, toàn bộ 18 Test Case Login đã được thực thi thành công và đạt trạng thái PASS.

Kết quả kiểm thử cho thấy chức năng đăng nhập đáp ứng các trường hợp kiểm thử đã được xây dựng, bao gồm validation dữ liệu đầu vào, xử lý thông tin đăng nhập không hợp lệ và chuyển hướng người dùng theo vai trò.

4.2.2. Admin Testing

Admin Testing được thực hiện nhằm kiểm tra các chức năng quản trị và chức năng thống kê của hệ thống. Nội dung kiểm thử tập trung vào Dashboard, theo dõi đơn hàng và bàn, quản lý món ăn, validation dữ liệu món ăn, quản lý nguyên liệu và kiểm tra các trường hợp API hoặc dữ liệu không hợp lệ.

Bộ Test Case Admin gồm 59 bản ghi kiểm thử, từ TC_ADMIN_001 đến TC_ADMIN_058. Trong dữ liệu Test Case có hai bản ghi cùng sử dụng mã TC_ADMIN_007, do đó tổng số bản ghi thực tế là 59.

Kết quả thực thi:

Trạng thái	Số lượng	Tỷ lệ
Pass	46	77,97%
Fail	13	22,03%
Skipped	0	0%
Blocked	0	0%
Not Run	0	0%
Tổng	59	100%

4.2.2.1. Dashboard và theo dõi đơn hàng

Các Test Case từ TC_ADMIN_001 đến TC_ADMIN_012 tập trung kiểm tra Dashboard và thông tin phục vụ quản lý đơn hàng, bàn và món ăn.

Các nội dung được kiểm tra bao gồm:

- Hiển thị các thông tin thống kê trên Dashboard.
- Truy vấn số liệu theo khoảng thời gian.
- Xử lý trường hợp khoảng thời gian không có dữ liệu.
- Phân trang danh sách món.
- Kiểm tra số lượng món đã bán.
- Kiểm tra doanh thu của món.
- Hiển thị các bàn đang có đơn chưa hoàn tất.
- Hiển thị thông tin món trong đơn.
- Hiển thị thời gian phục vụ.
- Hiển thị danh sách đơn/bàn đã hoàn tất.
- Cập nhật số lượng món đã phục vụ.
- Cập nhật trạng thái hoặc số liệu sau khi món hoàn thành.

Các Test Case trong nhóm này đều đạt trạng thái PASS. Kết quả cho thấy các chức năng Dashboard và theo dõi trạng thái đơn hàng trong phạm vi kiểm thử hoạt động đúng với Expected Result.

4.2.2.2. Quản lý món ăn

Các Test Case từ TC_ADMIN_013 đến TC_ADMIN_038 tập trung vào chức năng quản lý món ăn và validation dữ liệu.

Các chức năng chính được kiểm tra gồm:

- Hiển thị danh sách Food.
- Hiển thị danh sách đồ uống.
- Hiển thị danh sách Combo.
- Thêm món ăn.
- Cập nhật thông tin món.
- Xóa món.
- Kiểm tra tên món bắt buộc.
- Kiểm tra đường dẫn hình ảnh.
- Kiểm tra danh mục.
- Kiểm tra đơn giá.
- Kiểm tra đơn giá âm.
- Kiểm tra đơn giá bằng 0.
- Kiểm tra ký tự đặc biệt trong đơn giá.
- Kiểm tra URL không hợp lệ.
- Kiểm tra tên món rỗng.
- Kiểm tra độ dài dữ liệu.
- Kiểm tra dữ liệu nguyên liệu bằng 0 hoặc âm.

Trong nhóm này xuất hiện nhiều Test Case FAIL, chủ yếu liên quan đến validation dữ liệu đầu vào.

Các lỗi đáng chú ý gồm:

Test Case	Nội dung	Expected Result	Actual Result
TC_ADMIN_020	Dữ liệu món không hợp lệ	Hiển thị validation và không tạo món	Không báo lỗi validation và vẫn tạo món
TC_ADMIN_024	Không chọn danh mục	Yêu cầu chọn danh mục	Hệ thống mặc định danh mục Food
TC_ADMIN_028	Đơn giá âm	Từ chối đơn giá âm	Không báo lỗi và vẫn tạo món
TC_ADMIN_029	Đơn giá bằng 0	Từ chối đơn giá bằng 0	Hệ thống chấp nhận
TC_ADMIN_031	URL không hợp lệ	Từ chối URL	Hệ thống không từ chối
TC_ADMIN_032	URL không hợp lệ	Từ chối URL	Hệ thống không từ chối
TC_ADMIN_033	Tên món rỗng	Yêu cầu nhập tên hợp lệ	Tên món vẫn được chấp nhận
TC_ADMIN_036	Dữ liệu không hợp lệ	Từ chối lưu	Hệ thống vẫn cho phép tạo món
TC_ADMIN_037	Số lượng nguyên liệu bằng 0	Từ chối giá trị 0	Hệ thống chấp nhận và tự điều chỉnh thành 0.01
TC_ADMIN_038	Số lượng nguyên liệu âm	Từ chối giá trị âm	Hệ thống chấp nhận

Những kết quả trên cho thấy một số quy tắc validation chưa được kiểm soát đầy đủ ở tầng giao diện hoặc xử lý nghiệp vụ. Đây là nhóm vấn đề cần được xem xét trong quá trình cải thiện chất lượng hệ thống.

4.2.2.3. Quản lý kho và nguyên liệu

Các Test Case từ TC_ADMIN_039 đến TC_ADMIN_051 tập trung kiểm tra chức năng quản lý tồn kho, nhập kho và theo dõi lượng nguyên liệu tiêu thụ.

Các nội dung được kiểm tra gồm:

- Hiển thị thông tin nguyên liệu.
- Hiển thị trạng thái tồn kho.
- Cảnh báo nguyên liệu cần nhập thêm.
- Tạo phiếu nhập và cập nhật tồn kho.
- Lọc phiếu nhập theo nhà cung cấp.
- Lọc phiếu nhập theo khoảng thời gian.
- Lọc phiếu nhập theo nguyên liệu.
- Xóa điều kiện tìm kiếm.
- Hiển thị lịch sử tiêu thụ nguyên liệu.

- Lọc dữ liệu tiêu thụ theo thời gian.
- Kiểm tra số lượng nguyên liệu tiêu thụ.
- Xử lý trường hợp không có dữ liệu.

Các Test Case từ TC_ADMIN_039 đến TC_ADMIN_051 đều đạt trạng thái PASS.

Kết quả cho thấy các chức năng hiển thị, tìm kiếm, lọc dữ liệu và cập nhật tồn kho trong phạm vi được kiểm thử hoạt động phù hợp với Expected Result.

4.2.2.4. Kiểm thử xử lý lỗi và API

Các Test Case TC_ADMIN_052 đến TC_ADMIN_058 tập trung kiểm tra khả năng xử lý các tình huống lỗi và dữ liệu bất thường.

Các trường hợp được kiểm tra gồm:

- API hoặc thao tác hệ thống phát sinh lỗi.
- Không để giao diện bị crash khi xảy ra lỗi.
- Không hiển thị dữ liệu sai.
- Xử lý lỗi khi thêm món.
- Xử lý lỗi khi cập nhật món.
- Xử lý lỗi khi xóa món.
- Xử lý lỗi liên quan đến tồn kho.
- Xử lý lỗi khi tạo phiếu nhập.

Trong nhóm này, TC_ADMIN_054, TC_ADMIN_055 và TC_ADMIN_056 đều FAIL.

Các Actual Result cho thấy trong một số trường hợp thao tác thất bại nhưng giao diện vẫn hiển thị thông báo thành công. Cụ thể:

- TC_ADMIN_054: thao tác thêm món thất bại nhưng vẫn hiển thị thông báo thêm thành công.
- TC_ADMIN_055: thao tác cập nhật thất bại nhưng vẫn hiển thị thông báo sửa thành công.
- TC_ADMIN_056: thao tác xóa thất bại nhưng vẫn hiển thị thông báo thành công.

Đây là vấn đề về việc đồng bộ giữa kết quả xử lý Backend/API và thông báo hiển thị trên Frontend.

4.2.2.5. Tổng hợp kết quả Admin Testing

Kết quả thực thi 59 bản ghi Test Case Admin được tổng hợp như sau:

Nhóm chức năng	Kết quả chính
Dashboard và theo dõi đơn hàng	Các Test Case đạt PASS
Quản lý món ăn	Có cả PASS và FAIL
Validation dữ liệu món ăn	Phát hiện nhiều trường hợp FAIL
Quản lý kho và nguyên liệu	Các Test Case đạt PASS
Xử lý lỗi/API	Phát hiện một số trường hợp FAIL

Tổng cộng có 46 Test Case PASS và 13 Test Case FAIL, tương ứng tỷ lệ Pass 77,97%.

Các lỗi chủ yếu tập trung vào validation dữ liệu đầu vào và việc hiển thị thông báo thành công khi thao tác Backend/API thực tế không thành công. Những vấn đề này được ghi nhận để tiếp tục phân tích trong phần Defect và đánh giá chất lượng hệ thống.

4.2.3. Order and Cart Testing

Order and Cart Testing được thực hiện nhằm kiểm tra quá trình lựa chọn món ăn, quản lý giỏ hàng và đặt món của người dùng. Bộ Test Case tập trung vào các thao tác trên menu, popup thêm món, giỏ hàng và quá trình tạo đơn hàng.

Tổng cộng có 39 Test Case, bao phủ các trường hợp chức năng thông thường, dữ liệu không hợp lệ và các tình huống bất thường trong quá trình đặt món.

4.2.3.1. Kiểm tra thêm món và popup

Các Test Case từ TC_ORDER_001 đến TC_ORDER_008 tập trung kiểm tra quá trình lựa chọn món và thêm sản phẩm vào giỏ hàng.

Các chức năng được kiểm tra gồm:

- Mở popup thêm món.
- Đóng popup mà không thêm món.
- Tăng số lượng món.
- Giảm số lượng món.
- Không cho phép giảm số lượng xuống dưới giá trị tối thiểu.
- Thêm một món vào giỏ hàng.
- Thêm nhiều món khác nhau.
- Thêm cùng một món nhiều lần.

Các Test Case trong nhóm này chủ yếu đạt PASS, cho thấy các thao tác cơ bản trên popup và quá trình thêm món vào giỏ hàng hoạt động đúng theo Expected Result.

4.2.3.2. Kiểm tra giỏ hàng

Các Test Case từ TC_ORDER_009 đến TC_ORDER_021 tập trung kiểm tra các thao tác và dữ liệu trong giỏ hàng.

Các nội dung được kiểm tra gồm:

- Mở giỏ hàng.
- Tăng số lượng sản phẩm trong giỏ.
- Giảm số lượng sản phẩm.
- Xóa sản phẩm.
- Kiểm tra tổng tiền.
- Đặt món thành công.
- Kiểm tra trường hợp giỏ hàng trống.
- Kiểm tra badge số lượng sản phẩm.
- Kiểm tra thông tin đơn hàng.
- Kiểm tra trạng thái giỏ hàng sau khi đặt món.

Các Test Case trong nhóm này đều đạt trạng thái PASS, cho thấy các chức năng cơ bản của giỏ hàng và quá trình đặt món đáp ứng kết quả mong đợi.

4.2.3.3. Kiểm tra dữ liệu không hợp lệ và điều kiện biên

Các Test Case từ TC_ORDER_022 đến TC_ORDER_026 kiểm tra khả năng xử lý các trường hợp không hợp lệ.

Các trường hợp được kiểm tra gồm:

- Không thêm món khi số lượng không hợp lệ.
- Không cho phép thêm món đã hết hàng.
- Món đã được thêm vào giỏ nhưng sau đó hết hàng.
- Không cho phép đặt món khi giỏ hàng trống.
- Kiểm tra hành vi của hệ thống khi sản phẩm không còn khả dụng.

Trong nhóm này, TC_ORDER_022 và TC_ORDER_024 có trạng thái FAIL.

Kết quả cho thấy một số điều kiện nghiệp vụ liên quan đến số lượng và trạng thái hết hàng chưa được xử lý hoàn toàn theo Expected Result. Đây là những trường hợp cần được phân tích và ghi nhận để cải thiện validation và kiểm soát trạng thái sản phẩm.

4.2.3.4. Kiểm tra lỗi trong quá trình đặt món

Các Test Case từ TC_ORDER_027 đến TC_ORDER_031 tập trung kiểm tra các tình huống lỗi hoặc thao tác bất thường trong quá trình đặt món.

Các nội dung gồm:

- Đặt món khi mất kết nối mạng.
- Đặt món khi server không phản hồi.
- Ngăn việc tạo cùng một đơn hàng nhiều lần.
- Thay đổi số lượng sản phẩm liên tục.
- Xóa toàn bộ sản phẩm khỏi giỏ trước khi đặt món.

TC_ORDER_027 có trạng thái Skipped, trong khi các Test Case còn lại trong nhóm đạt trạng thái PASS.

Việc kiểm tra các tình huống này giúp đánh giá khả năng hệ thống xử lý các thao tác bất thường và hạn chế việc tạo dữ liệu đơn hàng không mong muốn.

4.2.3.5. Kiểm tra tính nhất quán dữ liệu

Các Test Case từ TC_ORDER_032 đến TC_ORDER_039 tập trung vào tính nhất quán của giỏ hàng và đơn hàng khi dữ liệu hoặc trạng thái hệ thống thay đổi.

Các trường hợp được kiểm tra gồm:

- Giá sản phẩm thay đổi trước khi đặt món.
- Refresh trang khi popup đang mở.
- Refresh trang sau khi thêm món vào giỏ.
- Phiên đăng nhập hết hạn khi đặt món.
- Đặt món với số lượng không hợp lệ.
- Kiểm tra giỏ hàng sau khi đặt món thất bại.
- Sản phẩm bị xóa khỏi hệ thống trong khi vẫn nằm trong giỏ.
- Kiểm tra badge khi thêm và xóa sản phẩm liên tục.

Trong nhóm này:

- TC_ORDER_032 — FAIL.
- TC_ORDER_034 — FAIL.
- TC_ORDER_035 — Skipped.
- TC_ORDER_036 — FAIL.

- Các Test Case còn lại — PASS.

Các kết quả này cho thấy vẫn còn một số vấn đề liên quan đến việc đồng bộ dữ liệu giữa giỏ hàng và trạng thái hiện tại của sản phẩm, cũng như xử lý các điều kiện đặc biệt khi đặt món.

4.2.3.6. Tổng hợp kết quả Order and Cart Testing

Trạng thái	Số lượng	Tỷ lệ
Pass	32	82,05%
Fail	5	12,82%
Skipped	2	5,13%
Blocked	0	0%
Not Run	0	0%
Tổng	39	100%

Tỷ lệ Test Case Pass:

$$32 / 39 \times 100\% = 82,05\%$$

Các Test Case Fail tập trung tại:

- Kiểm tra số lượng món không hợp lệ.
- Kiểm tra món hết hàng.
- Kiểm tra giá sản phẩm thay đổi trước khi đặt món.
- Kiểm tra trạng thái giỏ hàng sau khi refresh.
- Kiểm tra số lượng đặt món không hợp lệ.

Hai Test Case được Skipped liên quan đến tình huống mất kết nối mạng và phiên đăng nhập hết hạn.

Nhìn chung, các chức năng cơ bản của Order và Cart hoạt động đúng trong phần lớn trường hợp kiểm thử. Tuy nhiên, một số tình huống bất thường liên quan đến validation, trạng thái sản phẩm và tính nhất quán dữ liệu vẫn cần được cải thiện.

4.2.4. Kitchen Testing

Kitchen Testing được thực hiện nhằm kiểm tra các chức năng phục vụ quá trình tiếp nhận, theo dõi và xử lý món ăn tại khu vực Bếp. Nội dung kiểm thử tập trung vào việc hiển thị đơn hàng, cập nhật trạng thái món, xử lý nhiều đơn hàng đồng thời và khả năng đồng bộ trạng thái giữa khu vực Bếp và các bộ phận liên quan.

Bộ Test Case Kitchen gồm 38 Test Case, từ TC_KITCHEN_001 đến TC_KITCHEN_038.

4.2.4.1. Kiểm tra hiển thị và xử lý đơn hàng

Các Test Case đầu tiên tập trung kiểm tra việc hiển thị đơn hàng tại giao diện Bếp và khả năng thực hiện các thao tác xử lý món.

Các nội dung được kiểm tra gồm:

- Hiển thị danh sách đơn hàng cần xử lý.
- Hiển thị thông tin món trong đơn.

- Kiểm tra số lượng món.
- Kiểm tra trạng thái món.
- Cập nhật trạng thái món.
- Hoàn thành món.
- Xử lý nhiều món trong cùng một đơn.
- Kiểm tra đơn hàng sau khi trạng thái được cập nhật.

Kết quả kiểm thử cho thấy các thao tác xử lý món trong phạm vi được kiểm tra hoạt động đúng với Expected Result.

4.2.4.2. Kiểm tra nhiều đơn hàng và xử lý đồng thời

Một số Test Case được xây dựng để kiểm tra khả năng xử lý khi có nhiều đơn hàng xuất hiện đồng thời tại khu vực Bếp.

Các nội dung kiểm tra bao gồm:

- Nhiều đơn hàng được tạo liên tiếp.
- Nhiều món thuộc các đơn hàng khác nhau.
- Cập nhật trạng thái của từng món độc lập.
- Đảm bảo thao tác trên một đơn không ảnh hưởng đến đơn khác.
- Kiểm tra dữ liệu sau khi nhiều thao tác được thực hiện liên tiếp.

Kết quả thực thi cho thấy hệ thống xử lý các trường hợp kiểm thử đồng thời trong phạm vi Test Case mà không phát hiện Test Case Fail.

4.2.4.3. Kiểm tra đồng bộ trạng thái với Thu ngân

Kitchen Testing cũng kiểm tra khả năng đồng bộ trạng thái đơn hàng giữa khu vực Bếp và khu vực Thu ngân.

Các nội dung được kiểm tra gồm:

- Trạng thái món sau khi Bếp cập nhật.
- Trạng thái đơn hàng sau khi món hoàn thành.
- Khả năng cập nhật thông tin ở phía Thu ngân.
- Đảm bảo trạng thái được phản ánh phù hợp giữa các giao diện.
- Kiểm tra dữ liệu sau khi refresh.

Kết quả kiểm thử cho thấy các Test Case thuộc nhóm đồng bộ trạng thái đạt kết quả mong đợi.

4.2.4.4. Kiểm tra Refresh và Session

Các trường hợp kiểm thử cũng bao gồm các tình huống người dùng refresh trang hoặc phiên đăng nhập không còn hợp lệ.

Các nội dung được kiểm tra gồm:

- Refresh trang trong khi đang xử lý đơn.
- Refresh sau khi cập nhật trạng thái món.
- Kiểm tra dữ liệu sau khi refresh.
- Xử lý khi session hết hạn.
- Kiểm tra khả năng truy cập lại chức năng sau khi xác thực lại.

Những Test Case thuộc phạm vi này được sử dụng để đánh giá khả năng duy trì trạng thái và xử lý phiên người dùng của giao diện Bếp.

4.2.4.5. Kiểm tra lỗi API và mất kết nối

Một số Test Case được thiết kế để kiểm tra hành vi của hệ thống khi API không hoạt động bình thường hoặc kết nối mạng bị gián đoạn.

Các trường hợp bao gồm:

- API trả về lỗi.
- Request cập nhật trạng thái thất bại.
- Mất kết nối mạng trong quá trình thao tác.
- Server không phản hồi.
- Kiểm tra thông báo lỗi.
- Kiểm tra dữ liệu giao diện sau khi thao tác thất bại.

Các trường hợp này giúp đánh giá khả năng xử lý lỗi của giao diện Bếp và hạn chế việc hệ thống hiển thị trạng thái không chính xác.

4.2.4.6. Tổng hợp kết quả Kitchen Testing

Kết quả thực thi 38 Test Case được tổng hợp như sau:

Trạng thái	Số lượng	Tỷ lệ
Pass	36	94,74%
Fail	0	0%
Skipped	1	2,63%
Blocked	1	2,63%
Not Run	0	0%
Tổng	38	100%

Tỷ lệ Test Case Pass:

$$36 / 38 \times 100\% = 94,74\%$$

Không có Test Case nào có trạng thái FAIL trong bộ Kitchen Testing. Có 1 Test Case Skipped và 1 Test Case Blocked, cần được xem xét riêng để xác định điều kiện thực thi và khả năng thực hiện lại trong môi trường phù hợp.

Nhìn chung, các chức năng xử lý đơn hàng tại khu vực Bếp hoạt động ổn định trong phạm vi kiểm thử. Các chức năng hiển thị đơn hàng, cập nhật trạng thái món, xử lý nhiều đơn và đồng bộ trạng thái được kiểm tra với kết quả đạt yêu cầu đối với các Test Case đã thực thi thành công.

4.2.5. Payment and Table Testing

Payment and Table Testing được thực hiện nhằm kiểm tra quá trình thanh toán đơn hàng và quản lý trạng thái bàn trong hệ thống. Nội dung kiểm thử bao gồm kiểm tra thông tin hóa đơn, tổng tiền, các phương thức thanh toán, xử lý thanh toán thất bại và sự thay đổi trạng thái bàn trước và sau khi thanh toán.

Bộ Test Case Payment and Table gồm 40 Test Case.

4.2.5.1. Kiểm tra thông tin thanh toán

Các Test Case tập trung kiểm tra thông tin hiển thị trước và trong quá trình thanh toán.

Các nội dung được kiểm tra gồm:

- Hiện thị thông tin đơn hàng.
- Hiện thị danh sách món.
- Kiểm tra số lượng món.
- Kiểm tra đơn giá.
- Kiểm tra tổng tiền.
- Kiểm tra thông tin bàn.
- Kiểm tra thông tin hóa đơn trước khi thanh toán.

Mục tiêu của nhóm kiểm thử này là đảm bảo dữ liệu hiển thị cho nhân viên Thu ngân phù hợp với dữ liệu của đơn hàng.

4.2.5.2. Kiểm tra các phương thức thanh toán

Các Test Case tiếp theo tập trung vào việc thực hiện thanh toán bằng các phương thức được hệ thống hỗ trợ.

Các trường hợp được kiểm tra gồm:

- Thanh toán bằng phương thức hợp lệ.
- Kiểm tra trạng thái sau khi thanh toán thành công.
- Kiểm tra số tiền thanh toán.
- Kiểm tra hóa đơn sau thanh toán.
- Kiểm tra thao tác thanh toán lặp lại.
- Kiểm tra các trường hợp thanh toán không thành công.

Các Test Case được sử dụng để đảm bảo hệ thống chỉ cập nhật trạng thái đơn hàng và bàn khi quá trình thanh toán được xử lý thành công.

4.2.5.3. Kiểm tra trạng thái bàn

Một phần quan trọng của bộ Test Case là kiểm tra vòng đời trạng thái của bàn.

Các trường hợp được kiểm tra gồm:

- Bàn chưa có đơn.
- Bàn đang có đơn.
- Bàn có đơn đang được xử lý.
- Bàn sau khi hoàn tất thanh toán.
- Cập nhật trạng thái bàn sau khi thanh toán.
- Kiểm tra khả năng sử dụng lại bàn sau khi đơn hoàn tất.

Việc kiểm thử nhằm đảm bảo trạng thái bàn phản ánh đúng trạng thái hiện tại của đơn hàng và quá trình phục vụ.

4.2.5.4. Kiểm tra lỗi và tình huống bất thường

Các Test Case cũng kiểm tra khả năng xử lý lỗi trong quá trình thanh toán và quản lý bàn.

Các tình huống bao gồm:

- Mất kết nối mạng.
- API không phản hồi.
- Thanh toán thất bại.
- Dữ liệu đơn hàng không hợp lệ.

- Thực hiện thao tác khi trạng thái bàn không phù hợp.
- Refresh giao diện trong quá trình xử lý.
- Session hết hạn.
- Thực hiện nhiều thao tác liên tiếp.

Các trường hợp này nhằm đánh giá khả năng duy trì tính nhất quán của dữ liệu khi quá trình thanh toán hoặc cập nhật trạng thái gặp sự cố.

4.2.5.5. Tổng hợp kết quả Payment and Table Testing

Kết quả thực thi 40 Test Case được tổng hợp như sau:

Trạng thái	Số lượng	Tỷ lệ
Pass	32	80,00%
Fail	4	10,00%
Skipped	3	7,50%
Blocked	0	0%
Not Run	1	2,50%
Tổng	40	100%

Tỷ lệ Test Case Pass:

$$32 / 40 \times 100\% = 80,00\%$$

Có 4 Test Case FAIL, 3 Test Case Skipped và 1 Test Case Not Run. Các trường hợp này cần được phân tích riêng trong phần Defect và tổng hợp kết quả kiểm thử.

Kết quả cho thấy các chức năng thanh toán và quản lý bàn đã đáp ứng phần lớn các trường hợp kiểm thử được thực hiện. Tuy nhiên, vẫn tồn tại một số vấn đề liên quan đến xử lý lỗi, trạng thái hoặc điều kiện đặc biệt trong quá trình thanh toán cần được tiếp tục xem xét.

4.3. Unit Testing

Unit Testing được thực hiện do nhóm phát triển không thực hiện. Unit Testing nhằm kiểm tra các thành phần riêng lẻ của Backend, tập trung vào logic xử lý nghiệp vụ và các phương thức của hệ thống. Trong quá trình kiểm thử, các Test được thực hiện trên mã nguồn Backend và sử dụng JaCoCo để đánh giá mức độ bao phủ mã nguồn sau khi chạy Test.

JaCoCo cung cấp các chỉ số về Instruction Coverage, Branch Coverage, Line Coverage, Method Coverage và Class Coverage. Báo cáo được sử dụng để xác định mức độ mã nguồn đã được thực thi trong quá trình kiểm thử cũng như những khu vực chưa được bao phủ.

4.3.1. Phạm vi kiểm thử

Phạm vi kiểm thử Backend bao gồm các package chính:

- com.fastfood.service.impl
- com.fastfood.mapper
- com.fastfood.exception
- com.fastfood.repository

- com.fastfood.controller
- com.fastfood
- com.fastfood.config

Trong đó, service.impl và controller là hai nhóm thành phần có mức độ bao phủ cao và chứa phần lớn logic xử lý nghiệp vụ cũng như xử lý request của hệ thống.

4.3.2. Công cụ và phương pháp

JaCoCo (Java Code Coverage) được sử dụng để đo lường mức độ bao phủ mã nguồn sau khi thực hiện Test.

Các chỉ số chính gồm:

- Instruction Coverage: đo tỷ lệ instruction trong mã nguồn đã được thực thi.
- Branch Coverage: đo tỷ lệ các nhánh điều kiện đã được thực thi.
- Line Coverage: đo tỷ lệ dòng mã nguồn đã được thực thi.
- Method Coverage: đo tỷ lệ phương thức đã được thực thi.
- Class Coverage: đo tỷ lệ class đã được thực thi.

Việc sử dụng các chỉ số này giúp đánh giá không chỉ số lượng mã nguồn được thực thi mà còn khả năng kiểm thử các nhánh logic khác nhau trong chương trình.

4.3.3. Kết quả Coverage của Backend

Kết quả JaCoCo thu được sau quá trình thực hiện Test được tổng hợp như sau:

Chỉ số	Missed	Covered	Tổng	Coverage
Instructions	288	3.796	4.084	92%
Branches	97	259	356	72%
Lines	85	975	1.060	92%
Methods	9	180	189	95%
Classes	1	22	23	96%

Kết quả cho thấy mã nguồn Backend đạt 92% Instruction Coverage và 92% Line Coverage. Method Coverage đạt 95%, trong khi Class Coverage đạt 96%.

Branch Coverage đạt 72%, thấp hơn so với Instruction và Line Coverage. Điều này cho thấy phần lớn mã nguồn đã được thực thi nhưng vẫn còn một số nhánh điều kiện chưa được kiểm tra trong các Test hiện tại.

4.3.4. Kết quả theo package

Package	Instruction	Branch	Line	Method	Class
com.fastfood.service.impl	95%	78%	96%	100%	100%
com.fastfood.mapper	86%	51%	83%	100%	100%
com.fastfood.exception	6%	0%	5%	25%	100%
com.fastfood.repository	0%	0%	0%	0%	0%
com.fastfood.controller	98%	83%	98%	100%	100%
com.fastfood	37%	N/A	60%	50%	0%

com.fastfood.config	100%	N/A	100%	100%	100%
---------------------	------	-----	------	------	------

Package com.fastfood.controller đạt mức bao phủ cao với 98% Instruction Coverage, 83% Branch Coverage và 98% Line Coverage. Method Coverage và Class Coverage của package này đều đạt 100%.

Package com.fastfood.service.impl cũng có mức bao phủ cao, đạt 95% Instruction Coverage, 78% Branch Coverage và 96% Line Coverage. Method và Class Coverage đều đạt 100%.

Đối với com.fastfood.mapper, Instruction Coverage đạt 86% và Line Coverage đạt 83%. Tuy nhiên, Branch Coverage chỉ đạt 51%, cho thấy một số nhánh xử lý trong mapper chưa được kiểm tra đầy đủ.

Package com.fastfood.exception có mức bao phủ thấp với 6% Instruction Coverage, 0% Branch Coverage và 5% Line Coverage. Điều này cho thấy các trường hợp xử lý exception chưa được bao phủ đầy đủ.

Package com.fastfood.repository có 0% Coverage trong báo cáo hiện tại. Đây là khu vực cần được bổ sung Test nếu mục tiêu là nâng cao mức độ bao phủ toàn bộ Backend.

Package com.fastfood.config đạt 100% Instruction, Line, Method và Class Coverage.

4.3.5. Đánh giá kết quả Unit Testing

Kết quả Coverage cho thấy phần lớn mã nguồn Backend đã được thực thi trong quá trình kiểm thử. Instruction Coverage và Line Coverage đều đạt 92%, Method Coverage đạt 95% và Class Coverage đạt 96%.

Tuy nhiên, Branch Coverage chỉ đạt 72%, cho thấy vẫn tồn tại các nhánh điều kiện chưa được kiểm thử. Các khu vực có mức độ bao phủ thấp nhất là com.fastfood.repository và com.fastfood.exception.

Đặc biệt, package com.fastfood.repository có 0% Coverage và package com.fastfood.exception có Branch Coverage bằng 0%. Do đó, cần bổ sung thêm Test cho các phương thức Repository, các trường hợp Exception và các nhánh xử lý lỗi để tăng độ bao phủ.

Một điểm cần lưu ý là báo cáo JaCoCo trong phạm vi dữ liệu hiện có thể hiện Coverage của toàn bộ quá trình Test trên Backend, không phân tách riêng Coverage của Unit Test và Integration Test. Vì vậy, các giá trị 92%, 72%, 92%, 95% và 96% được sử dụng trong báo cáo như kết quả Coverage của bộ Test Backend, đây không phải là Coverage riêng của Unit Test.

4.4 Integration Testing

Integration Testing được thực hiện nhằm kiểm tra khả năng phối hợp giữa các thành phần của hệ thống sau khi từng thành phần đã được phát triển và kiểm thử. Đối với FastFood Management System, Integration Testing tập trung vào việc kiểm tra sự trao đổi dữ liệu giữa các lớp Backend cũng như sự phối hợp giữa Backend, Frontend và cơ sở dữ liệu trong các luồng nghiệp vụ chính. Do báo cáo JaCoCo được sử dụng trong quá trình đánh giá là báo cáo Coverage của toàn bộ quá trình Test trên Backend và không phân tách riêng Unit Test với Integration Test, các chỉ số

Coverage trong phần này được sử dụng như số liệu đánh giá mức độ bao phủ mã nguồn chung, không được xem là Coverage riêng của Integration Testing.

4.4.1. Phạm vi Integration Testing

Phạm vi Integration Testing tập trung vào các mối quan hệ giữa các thành phần sau:

- Controller – Service: kiểm tra việc tiếp nhận request và truyền dữ liệu đến tầng xử lý nghiệp vụ.
- Service – Repository: kiểm tra Service thực hiện đúng các thao tác với dữ liệu thông qua Repository.
- Repository – Database: kiểm tra việc truy vấn và cập nhật dữ liệu trong cơ sở dữ liệu.
- Frontend – Backend: kiểm tra việc gửi request từ giao diện và xử lý response từ REST API.
- Authentication – Authorization: kiểm tra sự kết hợp giữa xác thực tài khoản và phân quyền.
- Order – Kitchen: kiểm tra sự liên kết giữa việc tạo đơn hàng và xử lý đơn tại Bếp.
- Order – Payment – Table: kiểm tra sự phối hợp giữa đơn hàng, thanh toán và trạng thái bàn.

4.4.2. Kiểm thử tích hợp giữa các tầng Backend

Backend của hệ thống được tổ chức theo nhiều tầng xử lý. Khi Client gửi một request, request được tiếp nhận tại Controller, sau đó được chuyển đến Service để xử lý nghiệp vụ. Khi cần thao tác dữ liệu, Service tương tác với Repository và Repository thực hiện truy cập cơ sở dữ liệu.

Luồng tích hợp tổng quát được mô tả:

Client → Controller → Service → Repository → Database

Sau khi xử lý hoàn tất, kết quả được trả ngược về Client:

Database → Repository → Service → Controller → Client

Integration Testing tập trung kiểm tra tính chính xác của dữ liệu khi đi qua các tầng này, đồng thời kiểm tra việc xử lý kết quả và lỗi phát sinh trong quá trình giao tiếp giữa các thành phần.

4.4.3. Kiểm thử tích hợp Frontend và Backend

Frontend của hệ thống sử dụng ReactJS, trong khi Backend cung cấp các REST API để xử lý nghiệp vụ và truy cập dữ liệu.

Quá trình tích hợp được kiểm tra theo chuỗi:

1. Người dùng thực hiện thao tác trên giao diện.
2. Frontend tạo HTTP Request.
3. Backend tiếp nhận Request tại Controller.
4. Service xử lý nghiệp vụ.
5. Repository thực hiện thao tác với Database khi cần.
6. Backend trả HTTP Response.
7. Frontend xử lý Response và cập nhật giao diện.

Các chức năng được kiểm tra thông qua các luồng nghiệp vụ bao gồm đăng nhập, quản lý món, đặt món, xử lý đơn hàng, quản lý Bếp, thanh toán và quản lý bàn.

Việc kiểm thử các luồng này giúp phát hiện những lỗi chỉ xuất hiện khi nhiều thành phần hoạt động cùng nhau, chẳng hạn dữ liệu gửi từ giao diện không phù hợp với Backend, Backend xử lý thành công nhưng Frontend hiển thị sai hoặc trạng thái dữ liệu không được cập nhật đồng bộ.

4.4.4. Kiểm thử tích hợp Authentication và Authorization

Authentication và Authorization được kiểm tra thông qua luồng đăng nhập và phân quyền người dùng.

Sau khi người dùng gửi thông tin username và password, Backend thực hiện xác thực tài khoản. Nếu xác thực thành công, hệ thống xác định role của tài khoản và cho phép người dùng truy cập chức năng tương ứng.

Trong bộ Manual Functional Testing, chức năng Login có 18 Test Case, trong đó cả 18 Test Case đều đạt trạng thái PASS.

Các role được kiểm tra gồm:

Role	Chức năng sau khi đăng nhập
ADMIN	/admin/dashboard
ThuNgan	/payment
Bep	/kitchen
Ban01	/menu
Ban02	/menu
Ban03	/menu
Ban04	/menu

Ngoài đăng nhập hợp lệ, các Test Case còn kiểm tra dữ liệu đăng nhập không hợp lệ và một số dữ liệu đầu vào bất thường. Kết quả này cho thấy luồng Authentication và điều hướng theo Role hoạt động đúng trong phạm vi các trường hợp đã kiểm thử.

4.4.5. Kiểm thử tích hợp Order và Kitchen

Luồng Order và Kitchen kiểm tra sự phối hợp giữa quá trình tạo đơn của người dùng và quá trình tiếp nhận, xử lý đơn tại Bếp.

Luồng nghiệp vụ được thực hiện theo trình tự:

Customer/Table → Order → Backend → Kitchen → cập nhật trạng thái Order

Sau khi người dùng hoàn thành việc đặt món, dữ liệu đơn hàng được xử lý tại Backend và được sử dụng tại khu vực Bếp. Khi Bếp cập nhật trạng thái món, trạng thái tương ứng của đơn hàng được cập nhật trên hệ thống.

Trong Manual Functional Testing:

- Order and Cart: 39 Test Case, trong đó 32 PASS, 5 FAIL và 2 Skipped.
- Kitchen: 38 Test Case, trong đó 36 PASS, 1 Skipped và 1 Blocked.

Kết quả cho thấy phần lớn các trường hợp trong hai module hoạt động đúng. Tuy nhiên, một số trường hợp Fail ở Order và các trường hợp Skipped/Blocked cho thấy vẫn còn những điều kiện đặc biệt cần được kiểm tra hoặc thực hiện lại.

4.4.6. Kiểm thử tích hợp Order, Payment và Table

Một luồng tích hợp quan trọng khác là quá trình từ đặt món đến thanh toán và cập nhật trạng thái bàn.

Luồng xử lý được mô tả:

Order → Payment → cập nhật trạng thái Order → cập nhật trạng thái Table

Khi đơn hàng được tạo, hệ thống lưu thông tin đơn và các món tương ứng. Sau khi quá trình phục vụ hoàn tất, nhân viên thực hiện thanh toán. Kết quả thanh toán được sử dụng để cập nhật trạng thái của đơn hàng và bàn.

Bộ Payment and Table Testing gồm 40 Test Case, với kết quả:

- 32 PASS
- 4 FAIL
- 3 Skipped
- 1 Not Run

Các kết quả này cho thấy phần lớn luồng thanh toán và quản lý bàn đáp ứng Expected Result, nhưng vẫn tồn tại một số trường hợp cần được phân tích thêm.

4.4.7. Kiểm thử tích hợp với cơ sở dữ liệu

Database là thành phần được sử dụng bởi các chức năng quản lý món, nguyên liệu, đơn hàng, thanh toán và bàn. Integration Testing cần đảm bảo dữ liệu được truyền từ tầng nghiệp vụ xuống Repository và được lưu trữ hoặc truy vấn chính xác.

Các thao tác được xem xét gồm:

- Thêm dữ liệu món ăn.
- Cập nhật thông tin món.
- Xóa món.
- Truy vấn danh sách món.
- Cập nhật tồn kho.
- Tạo và cập nhật đơn hàng.
- Cập nhật trạng thái món.
- Cập nhật trạng thái bàn.
- Lưu thông tin thanh toán.

Trong quá trình Manual Testing, một số trường hợp liên quan đến validation dữ liệu đã phát hiện hệ thống vẫn chấp nhận một số dữ liệu không hợp lệ. Điều này cho thấy cần tăng cường kiểm soát dữ liệu trước khi thực hiện thao tác lưu hoặc cập nhật.

4.4.8. Kết quả Coverage liên quan đến Integration Testing

Báo cáo JaCoCo của Backend ghi nhận kết quả Coverage chung của toàn bộ quá trình Test:

Chỉ số	Coverage
Instruction Coverage	92%
Branch Coverage	72%
Line Coverage	92%

Method Coverage	95%
Class Coverage	96%

Các chỉ số trên cho thấy phần lớn mã nguồn Backend đã được thực thi trong quá trình Test. Tuy nhiên, vì JaCoCo không phân tách nguồn Coverage thành Unit Test và Integration Test trong báo cáo hiện có, các giá trị này không được sử dụng để kết luận riêng về tỷ lệ Coverage của Integration Testing.

Branch Coverage đạt 72% cũng cho thấy một số nhánh xử lý chưa được thực thi đầy đủ. Đây là cơ sở để tiếp tục bổ sung các trường hợp kiểm thử cho những luồng nghiệp vụ và tình huống lỗi chưa được bao phủ.

4.4.9. Đánh giá Integration Testing

Kết quả kiểm thử cho thấy các thành phần chính của hệ thống có khả năng phối hợp để thực hiện các nghiệp vụ chính, bao gồm:

- Authentication và Authorization.
- Quản lý món và dữ liệu.
- Đặt món và giỏ hàng.
- Xử lý đơn hàng tại Bếp.
- Thanh toán.
- Quản lý trạng thái bàn.
- Trao đổi dữ liệu giữa Frontend, Backend và Database.

Tuy nhiên, các Test Case Fail trong Order, Admin và Payment cho thấy vẫn tồn tại một số vấn đề trong validation, xử lý lỗi và đồng bộ trạng thái giữa các thành phần.

Do đó, Integration Testing cho thấy hệ thống đã thực hiện được phần lớn các luồng nghiệp vụ chính, nhưng cần tiếp tục bổ sung Test cho các trường hợp lỗi, dữ liệu không hợp lệ và các nhánh điều kiện chưa được bao phủ nhằm nâng cao tính ổn định và nhất quán của hệ thống.

4.5. Automation Testing

4.5.1. Mục tiêu và phạm vi Automation Testing

Automation Testing được thực hiện nhằm tự động hóa việc kiểm tra các luồng nghiệp vụ chính của hệ thống FastFood Management System thông qua giao diện người dùng. Selenium WebDriver được sử dụng để mô phỏng các thao tác của người dùng trên trình duyệt, kết hợp với TestNG để tổ chức và thực thi các Test Case nhằm tiết kiệm thời gian thực hiện Regression Testing.

Phạm vi Automation Testing trong báo cáo bao gồm ba nhóm chức năng chính:

- E2E Login: kiểm tra quá trình đăng nhập với thông tin hợp lệ và không hợp lệ.
- E2E Cart: kiểm tra khả năng thêm món ăn vào giỏ hàng.
- E2E Order: kiểm tra luồng nhiều người dùng đặt món đồng thời.

Bộ Automation Test hiện tại gồm 4 E2E Test Case, được phân chia như sau:

Nhóm kiểm thử	Số Test Case
E2E Login	2

E2E Cart	1
E2E Order	1
Tổng cộng	4

Các Test Case được thực hiện trong môi trường Local trên trình duyệt Google Chrome.

4.5.2. Công cụ và phương pháp thực hiện Automation Testing

Automation Testing được xây dựng bằng Java và sử dụng Selenium WebDriver để điều khiển trình duyệt Google Chrome. TestNG được sử dụng để quản lý Test Case và thực hiện quá trình kiểm thử tự động.

Các công cụ và kỹ thuật được sử dụng gồm:

Thành phần	Công cụ/Kỹ thuật
Ngôn ngữ lập trình	Java
Automation Framework	Selenium WebDriver
Test Framework	TestNG
Build Tool	Maven
Trình duyệt	Google Chrome
Thiết kế Test	Page Object Model
Quản lý Test Data	Data-driven Testing
Báo cáo kết quả	Excel Test Report

Page Object Model

Page Object Model (POM) được sử dụng để tách phần định nghĩa các thành phần trên giao diện khỏi phần thực hiện Test Case. Mỗi trang hoặc chức năng được biểu diễn thông qua một Page Class, trong đó chứa các locator và phương thức thao tác tương ứng.

Ví dụ, chức năng Login được xây dựng thông qua LoginPage, giúp Test Case có thể sử dụng các phương thức như đăng nhập mà không cần trực tiếp xử lý locator trong từng Test Case.

Cách tổ chức này giúp:

- Tái sử dụng các thao tác trên giao diện.
- Giảm sự lặp lại trong mã nguồn Test.
- Dễ dàng bảo trì khi giao diện thay đổi.
- Tách biệt phần xử lý giao diện và phần kiểm tra kết quả.

Data-driven Testing

Automation Test sử dụng dữ liệu kiểm thử được tách khỏi logic thực thi. Cách tiếp cận này cho phép cùng một Test Method được thực hiện với nhiều bộ dữ liệu khác nhau.

Đối với Login Testing, dữ liệu bao gồm tài khoản hợp lệ và các trường hợp đăng nhập không hợp lệ. Kết quả mong đợi cũng được sử dụng để xác định URL hoặc thông báo lỗi tương ứng sau khi thực hiện đăng nhập.

WebDriver Wait

Trong quá trình thực thi Selenium Test, cơ chế chờ được sử dụng để đảm bảo phần tử trên giao diện đã sẵn sàng trước khi thực hiện thao tác.

Việc sử dụng WebDriverWait giúp hạn chế lỗi xảy ra do Selenium thực hiện thao tác trước khi trang hoặc phần tử được tải hoàn tất.

Quy trình thực hiện Automation Test

Quy trình Automation Testing được thực hiện theo các bước:

1. Khởi động Backend và Frontend của hệ thống.
2. Khởi tạo trình duyệt Google Chrome.
3. Mở URL của chức năng cần kiểm thử.
4. Đăng nhập bằng dữ liệu Test tương ứng.
5. Thực hiện các thao tác nghiệp vụ bằng Selenium WebDriver.
6. Kiểm tra Actual Result với Expected Result.
7. Ghi nhận trạng thái Test Case.
8. Xuất kết quả thực thi vào Test Report.

Phương pháp trên giúp tái hiện các thao tác của người dùng thực tế và kiểm tra kết quả của toàn bộ luồng nghiệp vụ thay vì chỉ kiểm tra từng thành phần riêng lẻ.

4.5.3. Chi tiết các Test Case Automation

Bộ Automation Test gồm 4 Test Case, bao phủ các chức năng Login, Add to Cart và Order. Các Test Case được thực hiện trên môi trường Local bằng trình duyệt Google Chrome.

4.5.3.1. Login Automation Testing

Login Automation Testing gồm 2 Test Case nhằm kiểm tra trường hợp đăng nhập thành công và đăng nhập với thông tin không hợp lệ.

E2E_LOGIN_001 – Login với tài khoản hợp lệ

- Test Class: LoginTests
- Test Method: loginValidAccount
- Mục tiêu: Kiểm tra khả năng đăng nhập bằng tài khoản hợp lệ.
- Test Data: Username và password hợp lệ.
- Expected Result: Người dùng đăng nhập thành công và được chuyển đến trang tương ứng với quyền của tài khoản.
- Actual Result: Người dùng đăng nhập thành công và được chuyển đến đúng trang.
- Status: PASS

E2E_LOGIN_002 – Login với tài khoản không hợp lệ

- Test Class: LoginTests
- Test Method: loginInvalidAccount
- Mục tiêu: Kiểm tra khả năng xử lý thông tin đăng nhập không hợp lệ.
- Test Data: Thông tin tài khoản không hợp lệ.
- Expected Result: Hệ thống không cho phép đăng nhập và hiển thị thông báo lỗi phù hợp.
- Actual Result: Hệ thống từ chối đăng nhập và hiển thị thông báo lỗi.
- Status: PASS

Hai Test Case cho thấy chức năng Login có thể được kiểm tra tự động đối với cả trường hợp hợp lệ và không hợp lệ.

4.5.3.2. Add to Cart Automation Testing

E2E_CART_001 – Thêm món vào giỏ hàng

- Test Class: AddToCartTest
- Test Method: addFoodToCart
- Mục tiêu: Kiểm tra khả năng thêm một món ăn vào giỏ hàng.
- Pre-condition: Người dùng đã đăng nhập và đang ở giao diện Menu.
- Test Data: Đùi gà chiên giòn.
- Expected Result: Món được thêm thành công vào giỏ hàng và thông tin giỏ hàng được cập nhật.
- Actual Result: Món được thêm thành công vào giỏ hàng.
- Status: PASS

Test Case này mô phỏng thao tác thực tế của người dùng từ việc truy cập Menu, lựa chọn món ăn và kiểm tra kết quả sau khi thêm món.

4.5.3.3. Order Automation Testing

E2E_ORDER_001 – Nhiều người dùng đặt món

- Test Class: MultiUserOrderTest
- Test Method: createOrderMultiUser
- Mục tiêu: Kiểm tra khả năng tạo đơn hàng cho nhiều người dùng/bàn.
- Pre-condition: Hệ thống đang hoạt động và các tài khoản/bàn kiểm thử đã được chuẩn bị.
- Test Data: Ban01, Ban02.
- Expected Result: Các đơn hàng được tạo thành công và dữ liệu của từng bàn được xử lý đúng.
- Actual Result: Các đơn hàng được tạo thành công.
- Status: PASS

Test Case này giúp kiểm tra không chỉ chức năng tạo đơn mà còn khả năng xử lý các đơn hàng được tạo từ nhiều người dùng/bàn khác nhau.

4.5.3.4. Tổng hợp các Test Case

Test Case ID	Chức năng	Test Class	Test Method	Status
E2E_LOGIN_001	Login hợp lệ	LoginTests	loginValidAccount	PASS
E2E_LOGIN_002	Login không hợp lệ	LoginTests	loginInvalidAccount	PASS
E2E_CART_001	Thêm món vào giỏ	AddToCartTest	addFoodToCart	PASS
E2E_ORDER_001	Nhiều người dùng đặt món	MultiUserOrderTest	createOrderMultiUser	PASS

4.5.4. Kết quả thực thi Automation Testing

Sau khi thực hiện các kịch bản Automation Test trên môi trường Local với trình duyệt Google Chrome, kết quả thực thi được ghi nhận trong Automation Test Report.

Tổng cộng có 4 Test Case được thực thi, bao gồm Login, Add to Cart và Order.

Trạng thái	Số lượng	Tỷ lệ
PASS	4	100%
FAIL	0	0%
Tổng cộng	4	100%

Tỷ lệ Pass được tính theo công thức:

Pass Rate = (Số Test Case PASS / Tổng số Test Case) × 100%

Pass Rate = (4 / 4) × 100% = 100%

Kết quả chi tiết:

Test Case	Chức năng	Kết quả
E2E_LOGIN_001	Đăng nhập hợp lệ	PASS
E2E_LOGIN_002	Đăng nhập không hợp lệ	PASS
E2E_CART_001	Thêm món vào giỏ hàng	PASS
E2E_ORDER_001	Nhiều người dùng đặt món	PASS

Kết quả cho thấy cả 4 Test Case Automation đều được thực thi thành công và không ghi nhận Test Case Fail trong lần thực thi hiện tại.

Đối với chức năng Login, cả trường hợp đăng nhập hợp lệ và không hợp lệ đều đạt PASS. Điều này cho thấy Automation Test có thể xác nhận được cả luồng thành công và luồng xử lý dữ liệu đăng nhập không hợp lệ.

Đối với chức năng Add to Cart, Test Case thêm món vào giỏ hàng đạt PASS, cho thấy Selenium có thể mô phỏng thành công thao tác lựa chọn sản phẩm và kiểm tra kết quả trên giao diện.

Đối với chức năng Order, Test Case kiểm tra nhiều người dùng đặt món đạt PASS. Kết quả này cho thấy luồng tạo đơn hàng trong kịch bản kiểm thử đã thực hiện đúng Expected Result.

Nhìn chung, kết quả 4/4 Test Case PASS cho thấy các kịch bản Automation hiện tại hoạt động ổn định trong môi trường và dữ liệu được sử dụng cho lần thực thi này.

Tuy nhiên, kết quả 100% Pass chỉ phản ánh 4 Test Case đã được Automation hóa và thực thi, không thể đại diện cho toàn bộ chức năng của hệ thống. Các module như Admin, Kitchen và Payment vẫn cần được bổ sung thêm Automation Test nếu muốn mở rộng phạm vi tự động hóa.

4.5.5. Đánh giá Automation Testing

Kết quả Automation Testing cho thấy bộ Test hiện tại đã tự động hóa được một số luồng nghiệp vụ quan trọng của hệ thống. Với 4/4 Test Case PASS, các kịch bản Login, Add to Cart và Order đều đạt kết quả mong đợi trong lần thực thi.

Automation Testing mang lại một số lợi ích cho quá trình kiểm thử:

- Giảm thời gian thực hiện các thao tác kiểm thử lặp lại.
- Có thể thực hiện lại Test Case nhanh chóng sau khi thay đổi mã nguồn.
- Hạn chế sai sót do thao tác thủ công.

- Hỗ trợ kiểm tra các luồng nghiệp vụ từ giao diện người dùng.
- Có thể tích hợp vào quy trình Regression Testing.
- Có thể mở rộng số lượng Test Case khi hệ thống phát triển.

Tuy nhiên, phạm vi Automation hiện tại còn giới hạn ở Login, Add to Cart và Order. Các chức năng Admin, Kitchen và Payment chưa được bao phủ đầy đủ bằng Automation Test. Vì vậy, cần tiếp tục xây dựng thêm các kịch bản Automation cho các chức năng này.

Bên cạnh đó, Automation Test phụ thuộc vào trạng thái của Frontend, Backend, Database, trình duyệt và dữ liệu kiểm thử. Khi giao diện hoặc locator thay đổi, các Test Script có thể cần được cập nhật để đảm bảo khả năng thực thi.

Như vậy, Automation Testing đã đạt kết quả tích cực trong phạm vi hiện tại, đồng thời tạo nền tảng để mở rộng Regression Test tự động cho toàn bộ hệ thống.

4.5.6. Kết luận Automation Testing

Automation Testing được triển khai bằng Selenium WebDriver, Java và TestNG nhằm tự động hóa các luồng nghiệp vụ trên giao diện hệ thống.

Bộ Test hiện tại gồm 4 E2E Test Case, bao gồm:

- 2 Test Case Login.
- 1 Test Case Add to Cart.
- 1 Test Case Order.

Kết quả thực thi đạt 4/4 PASS, tương đương 100% Pass Rate. Không có Test Case Fail trong lần thực thi hiện tại.

Kết quả này cho thấy các kịch bản Automation đã được xây dựng có khả năng kiểm tra thành công các luồng nghiệp vụ được lựa chọn. Tuy nhiên, phạm vi Automation cần tiếp tục được mở rộng sang các chức năng Admin, Kitchen và Payment để tăng mức độ bao phủ và hỗ trợ hiệu quả hơn cho Regression Testing.

4.6. Performance Testing

4.6.1. Mục tiêu kiểm thử hiệu năng

Performance Testing được thực hiện nhằm đánh giá khả năng đáp ứng của hệ thống FastFood Management System khi có nhiều yêu cầu được gửi đến Backend trong cùng một khoảng thời gian.

Mục tiêu của Performance Testing là:

- Đánh giá thời gian phản hồi của các API quan trọng.
- Kiểm tra khả năng xử lý nhiều request đồng thời.
- Đánh giá throughput của hệ thống.
- Xác định tỷ lệ request xảy ra lỗi trong quá trình kiểm thử.

- Kiểm tra tính ổn định của hệ thống khi thực hiện nhiều request liên tiếp.
- Phát hiện các vấn đề liên quan đến khả năng xử lý đồng thời và truy cập cơ sở dữ liệu.

JMeter được sử dụng để mô phỏng nhiều người dùng gửi request đến hệ thống. Trong phạm vi kiểm thử, hai API chính được lựa chọn là Login API và Orders API, tương ứng với các nghiệp vụ xác thực người dùng và xử lý đơn hàng.

4.6.2. Môi trường và cấu hình kiểm thử

Performance Testing được thực hiện bằng Apache JMeter trên môi trường Local. Hệ thống được kiểm thử thông qua các API Backend nhằm đánh giá khả năng xử lý request khi có nhiều người dùng đồng thời.

Cấu hình kiểm thử được sử dụng:

Chỉ tiêu	Giá trị
Công cụ	Apache JMeter
Concurrent Users	10
Loop Count	5
API được kiểm thử	Login API, Orders API
Tổng số request	100
Môi trường	Local
Error Rate	0%

Với 10 người dùng đồng thời và mỗi người thực hiện 5 vòng lặp, mỗi vòng thực hiện các request tương ứng với Login API và Orders API. Kết quả thu được tổng cộng 100 samples.

Việc lựa chọn Login API và Orders API nhằm kiểm tra hai nhóm nghiệp vụ quan trọng của hệ thống: xác thực người dùng và xử lý đơn hàng.

4.6.3. Kịch bản kiểm thử hiệu năng

Hai API được lựa chọn để thực hiện Performance Testing gồm Login API và Orders API.

Kịch bản 1: Login API

Login API được sử dụng để kiểm tra khả năng xử lý các yêu cầu đăng nhập đồng thời.

Các request được gửi với nhiều người dùng trong cùng thời điểm nhằm đánh giá:

- Thời gian phản hồi của API.
- Khả năng xử lý nhiều request đăng nhập.

- Tỷ lệ request xảy ra lỗi.
- Khả năng duy trì tính ổn định khi request được lặp lại.

Kết quả kiểm thử ghi nhận 50 samples, thời gian phản hồi trung bình 7 ms, thời gian nhỏ nhất 4 ms và lớn nhất 11 ms. Error Rate đạt 0%, cho thấy toàn bộ request Login API trong lần kiểm thử này được xử lý thành công.

Kịch bản 2: Orders API

Orders API được sử dụng để đánh giá khả năng xử lý các request liên quan đến đơn hàng khi có nhiều người dùng thực hiện đồng thời.

Các chỉ tiêu được quan sát gồm:

- Thời gian phản hồi.
- Throughput.
- Error Rate.
- Khả năng xử lý request liên tục.

Kết quả ghi nhận 50 samples, thời gian phản hồi trung bình 20 ms, nhỏ nhất 15 ms và lớn nhất 29 ms. Error Rate đạt 0%.

So với Login API, Orders API có thời gian phản hồi trung bình cao hơn do quá trình xử lý đơn hàng có nhiều thao tác nghiệp vụ và truy cập dữ liệu hơn.

4.6.4. Kết quả kiểm thử hiệu năng

Kết quả Performance Testing được thu thập bằng Apache JMeter sau khi thực hiện các request đồng thời đến Login API và Orders API. Tổng cộng có 100 requests được thực hiện, trong đó mỗi API nhận 50 requests.

Kết quả chi tiết được thể hiện trong bảng sau:

API	Samples	Average (ms)	Min (ms)	Max (ms)	Std. Dev. (ms)	Error	Throughput
Login API	50	7	4	11	1.52	0.00%	49.8 req/s
Orders API	50	20	15	29	3.44	0.00%	49.3 req/s
TOTAL	100	13	4	29	7.25	0.00%	97.8 req/s

Trong đó:

- Samples: tổng số request được thực hiện đối với từng API.

- Average: thời gian phản hồi trung bình của request.
- Min: thời gian phản hồi nhỏ nhất.
- Max: thời gian phản hồi lớn nhất.
- Std. Dev.: độ lệch chuẩn của thời gian phản hồi.
- Error: tỷ lệ request xảy ra lỗi.
- Throughput: số request hệ thống có thể xử lý trung bình trong một giây.

Đối với Login API, 50 request được thực hiện với thời gian phản hồi trung bình 7 ms, thấp nhất 4 ms và cao nhất 11 ms. Throughput đạt 49.8 requests/second và Error Rate bằng 0%.

Đối với Orders API, 50 request được thực hiện với thời gian phản hồi trung bình 20 ms, thấp nhất 15 ms và cao nhất 29 ms. Throughput đạt 49.3 requests/second và Error Rate bằng 0%.

Trên toàn bộ 100 request, thời gian phản hồi trung bình là 13 ms, thời gian thấp nhất 4 ms, cao nhất 29 ms, trong khi Error Rate bằng 0%. Throughput tổng thể đạt 97.8 requests/second.

4.6.5. Phân tích kết quả

Kết quả Performance Testing cho thấy hệ thống có khả năng xử lý các request Login API và Orders API ổn định trong cấu hình kiểm thử được thực hiện.

Login API có thời gian phản hồi trung bình 7 ms, thấp hơn đáng kể so với Orders API với 20 ms. Điều này cho thấy Login API có thời gian xử lý ngắn hơn trong môi trường kiểm thử.

Orders API có thời gian phản hồi cao hơn do thực hiện các xử lý liên quan đến nghiệp vụ đơn hàng. Tuy nhiên, thời gian phản hồi lớn nhất được ghi nhận là 29 ms, và không có request nào xảy ra lỗi.

Toàn bộ 100 requests đều được xử lý thành công, tương ứng với:

Success Rate = 100%

và:

Error Rate = 0%

Throughput tổng thể đạt 97.8 requests/second, cho thấy hệ thống có khả năng tiếp nhận và xử lý số lượng request tương đối lớn trong khoảng thời gian kiểm thử.

Độ lệch chuẩn của Login API là 1.52 ms, trong khi Orders API là 3.44 ms. Điều này cho thấy thời gian phản hồi của Login API có mức dao động nhỏ hơn Orders API trong lần kiểm thử.

Nhìn chung, trong phạm vi 10 concurrent users và 5 vòng lặp, hệ thống đáp ứng ổn định đối với hai API được kiểm thử. Không ghi nhận lỗi request trong lần chạy JMeter được sử dụng để đánh giá.

Cần lưu ý rằng kết quả này chỉ phản ánh hiệu năng của hệ thống trong môi trường và cấu hình kiểm thử cụ thể, với phạm vi API gồm Login và Orders. Vì vậy, chưa thể sử dụng kết quả này để kết luận về khả năng chịu tải tối đa của toàn bộ hệ thống.

4.6.6. Đánh giá Performance Testing

Kết quả kiểm thử cho thấy hệ thống đáp ứng tốt trong kịch bản có 10 người dùng đồng thời, mỗi người thực hiện 5 vòng lặp.

Các kết quả chính đạt được:

- Tổng số request: 100.
- Success Rate: 100%.
- Error Rate: 0%.
- Average Response Time: 13 ms.
- Maximum Response Time: 29 ms.
- Throughput: 97.8 requests/second.

Login API có thời gian phản hồi thấp và ổn định, trong khi Orders API có thời gian xử lý cao hơn nhưng vẫn duy trì Error Rate bằng 0%.

Trong lần kiểm thử được sử dụng làm kết quả chính thức, không ghi nhận lỗi Duplicate Key. Vì vậy, nội dung bản báo cáo cũ về lỗi Duplicate Key không được đưa vào kết luận của lần kiểm thử này. Nếu muốn phân tích lỗi Duplicate Key như một kết quả của Stress/Concurrency Testing riêng, cần có Test Run hoặc log tương ứng để chứng minh.

Nhìn chung, hệ thống đáp ứng ổn định trong phạm vi tải được kiểm thử. Tuy nhiên, để đánh giá khả năng chịu tải ở mức cao hơn, cần thực hiện thêm các kịch bản với số lượng Concurrent Users lớn hơn và thời gian chạy dài hơn.

4.7. Defect Management

4.7.1. Tổng quan Defect Management

Defect Management được thực hiện nhằm ghi nhận, phân loại, theo dõi và đánh giá các lỗi được phát hiện trong quá trình kiểm thử hệ thống. Các defect được xác định dựa trên sự khác biệt giữa Expected Result và Actual Result của Test Case.

Trong quá trình Manual Functional Testing, một số lỗi đã được phát hiện ở các module Admin, Order and Cart và Payment and Table. Các lỗi chủ yếu liên quan đến validation dữ liệu, xử lý lỗi API và cập nhật trạng thái trên giao diện.

4.7.2. Các nhóm defect được phát hiện

Các defect được phân thành các nhóm chính:

Nhóm defect	Mô tả
Validation	Hệ thống chấp nhận dữ liệu không hợp lệ
Business Logic	Kết quả xử lý không đúng với quy tắc nghiệp vụ
API/Error Handling	API thất bại nhưng giao diện vẫn thông báo thành công
Data Consistency	Dữ liệu giữa các trạng thái/module chưa đồng bộ
UI/Message	Thông báo trên giao diện không phản ánh đúng kết quả xử lý

4.7.3. Defect trong Admin Module

Admin Testing phát hiện 13 Test Case Fail trên tổng số 59 Test Case.

Các lỗi nổi bật liên quan đến:

- Hệ thống chấp nhận đơn giá âm.
- Hệ thống chấp nhận đơn giá bằng 0.
- URL hình ảnh không hợp lệ vẫn được chấp nhận.
- Tên món rỗng vẫn có thể được xử lý.
- Dữ liệu nguyên liệu bằng 0 hoặc âm chưa được validation đầy đủ.
- Một số thao tác API thất bại nhưng Frontend vẫn hiển thị thông báo thành công.

Các lỗi này cho thấy validation ở một số trường hợp chưa được kiểm soát đầy đủ và việc xử lý response từ Backend tại Frontend cần được cải thiện.

4.7.4. Defect trong Order and Cart

Order and Cart Testing có 5 Test Case Fail trên tổng số 39 Test Case.

Các lỗi được ghi nhận tập trung vào:

- Xử lý số lượng món không hợp lệ.
- Xử lý sản phẩm hết hàng.
- Giá sản phẩm thay đổi trong quá trình đặt món.

- Trạng thái giỏ hàng sau khi refresh.
- Kiểm tra tính hợp lệ của dữ liệu trước khi tạo đơn.

Các vấn đề này chủ yếu liên quan đến việc đồng bộ trạng thái sản phẩm và validation trong quá trình đặt món.

4.7.5. Defect trong Payment and Table

Payment and Table Testing có 4 Test Case Fail trên tổng số 40 Test Case.

Các lỗi được phát hiện liên quan đến quá trình thanh toán và cập nhật trạng thái bàn/đơn hàng. Đây là những vấn đề cần được ưu tiên xử lý vì trạng thái thanh toán có ảnh hưởng trực tiếp đến trạng thái của đơn hàng và bàn.

4.7.6. Đánh giá Defect

Từ kết quả kiểm thử có thể nhận thấy defect tập trung chủ yếu ở ba nhóm:

1. Validation dữ liệu đầu vào.
2. Xử lý lỗi giữa Frontend và Backend.
3. Đồng bộ trạng thái giữa các module.

Trong khi đó, Login và Kitchen có kết quả tốt hơn:

- Login: 18/18 PASS.
- Kitchen: 36/38 PASS, không có Test Case FAIL.
- Automation: 4/4 PASS.

Do đó, các defect cần ưu tiên khắc phục nằm ở các chức năng Admin, Order và Payment, đặc biệt là các lỗi có khả năng dẫn đến dữ liệu không hợp lệ hoặc trạng thái hệ thống không chính xác.

4.8. Tổng hợp và đánh giá kết quả kiểm thử

4.8.1. Tổng hợp kết quả kiểm thử

Sau khi thực hiện các mức kiểm thử gồm Manual Functional Testing, Unit Testing, Integration Testing, Automation Testing và Performance Testing, kết quả kiểm thử được tổng hợp để đánh giá chất lượng tổng thể của hệ thống FastFood Management System.

Các kết quả chính được tổng hợp như sau:

Phương pháp kiểm thử	Kết quả chính
Manual Functional Testing	Phát hiện các lỗi liên quan đến validation, nghiệp vụ và xử lý dữ liệu

Unit Testing	JaCoCo: 92% Instruction, 72% Branch, 92% Line, 95% Method, 96% Class
Integration Testing	Kiểm tra sự phối hợp giữa Controller, Service, Repository, Database và các luồng nghiệp vụ
Automation Testing	4/4 Test Case PASS – 100%
Performance Testing	100 requests, 0% Error, Average 13 ms, Throughput 97.8 req/s

Kết quả cho thấy hệ thống đã được kiểm thử ở nhiều mức khác nhau, từ kiểm tra chức năng riêng lẻ đến kiểm tra sự phối hợp giữa các thành phần và khả năng đáp ứng của hệ thống dưới tải.

4.8.2. Đánh giá kết quả Functional Testing

Manual Functional Testing được sử dụng để kiểm tra các chức năng chính của hệ thống từ góc nhìn người dùng.

Kết quả kiểm thử cho thấy các chức năng chính đã được kiểm tra, bao gồm:

- Login.
- Admin.
- Order and Cart.
- Kitchen.
- Payment and Table.

Trong đó, Login đạt 18/18 Test Case PASS. Kitchen có 36/38 Test Case PASS và không ghi nhận Test Case Fail. Một số lỗi vẫn được phát hiện ở Admin, Order and Cart và Payment and Table, chủ yếu liên quan đến validation dữ liệu, xử lý lỗi và đồng bộ trạng thái.

Các kết quả Fail trong Manual Testing có ý nghĩa quan trọng vì giúp xác định những khu vực cần tiếp tục sửa lỗi trước khi hệ thống được đưa vào sử dụng thực tế.

4.8.3. Đánh giá Code Coverage

Kết quả JaCoCo cho thấy mã nguồn Backend có mức độ bao phủ tương đối cao:

Chỉ số	Coverage
Instruction	92%
Branch	72%
Line	92%
Method	95%

Class	96%
-------	-----

Instruction Coverage và Line Coverage đều đạt 92%, cho thấy phần lớn mã nguồn Backend đã được thực thi trong quá trình Test.

Method Coverage đạt 95% và Class Coverage đạt 96%, cho thấy phần lớn phương thức và class đã được bao phủ.

Tuy nhiên, Branch Coverage chỉ đạt 72%. Một số package như com.fastfood.repository và com.fastfood.exception có mức Coverage thấp, do đó cần bổ sung thêm Test cho các nhánh xử lý lỗi và các phương thức chưa được thực thi.

Các chỉ số trên là Coverage chung của quá trình Test Backend, không được tách riêng thành Unit Coverage và Integration Coverage.

4.8.4. Đánh giá Automation Testing

Automation Testing được thực hiện bằng Selenium WebDriver và TestNG.

Bộ Automation Test hiện tại gồm 4 Test Case:

- 2 Test Case Login.
- 1 Test Case Add to Cart.
- 1 Test Case Order.

Kết quả:

4/4 Test Case PASS = 100%

Kết quả này cho thấy các luồng nghiệp vụ được tự động hóa đã thực thi thành công trong môi trường kiểm thử.

Tuy nhiên, phạm vi Automation hiện tại vẫn tập trung vào một số chức năng chính. Các module như Admin, Kitchen và Payment cần được tiếp tục tự động hóa để nâng cao khả năng Regression Testing.

4.8.5. Đánh giá Performance Testing

Performance Testing được thực hiện bằng JMeter với cấu hình 10 Concurrent Users và 5 Loop Count.

Kết quả tổng hợp:

Chỉ tiêu	Kết quả
Concurrent Users	10
Loop Count	5

Tổng Samples	100
Average Response Time	13 ms
Minimum Response Time	4 ms
Maximum Response Time	29 ms
Error Rate	0%
Throughput	97.8 requests/sec

Trong lần kiểm thử được sử dụng làm kết quả chính thức, toàn bộ 100 request được xử lý thành công. Thời gian phản hồi trung bình là 13 ms và thời gian phản hồi lớn nhất là 29 ms.

Kết quả cho thấy hệ thống đáp ứng ổn định trong phạm vi tải được kiểm thử. Tuy nhiên, cần thực hiện thêm các bài kiểm thử với số lượng người dùng đồng thời cao hơn nếu muốn đánh giá giới hạn chịu tải của hệ thống.

4.8.6. Các vấn đề còn tồn tại

Qua tổng hợp kết quả kiểm thử, một số vấn đề vẫn cần được cải thiện:

1. Validation dữ liệu chưa đầy đủ: một số trường hợp dữ liệu không hợp lệ vẫn được hệ thống chấp nhận.
2. Xử lý lỗi API: một số trường hợp Backend/API xử lý thất bại nhưng giao diện chưa phản ánh chính xác kết quả.
3. Branch Coverage chưa cao: Branch Coverage đạt 72%, cho thấy vẫn còn các nhánh logic chưa được kiểm thử.
4. Automation Coverage còn hạn chế: Automation Test mới tập trung vào Login, Cart và Order.
5. Một số Functional Test Case chưa đạt: các module Admin, Order and Cart và Payment and Table vẫn còn Test Case Fail.
6. Performance Testing còn giới hạn: mới kiểm tra với 10 người dùng đồng thời, chưa xác định được giới hạn chịu tải tối đa của hệ thống.

4.8.7. Đánh giá tổng thể

Nhìn chung, quá trình kiểm thử đã bao phủ nhiều khía cạnh của hệ thống, từ chức năng, mã nguồn, tích hợp, tự động hóa đến hiệu năng.

Hệ thống đạt kết quả tốt ở một số khía cạnh như:

- Code Coverage cao ở mức Instruction, Line, Method và Class.

- Login Functional Testing đạt 100% Pass.
- Automation Testing đạt 100% Pass trong phạm vi 4 Test Case.
- Performance Testing đạt 0% Error Rate với 100 request.
- Thời gian phản hồi trong Performance Testing ở mức thấp, với trung bình 13 ms.

Bên cạnh đó, các Test Case Fail và các package có Coverage thấp cho thấy hệ thống vẫn cần tiếp tục được cải thiện, đặc biệt ở các chức năng validation, xử lý lỗi, đồng bộ dữ liệu và các nhánh nghiệp vụ chưa được kiểm thử.

Do đó, kết quả kiểm thử cho thấy hệ thống đã đáp ứng được phần lớn các yêu cầu chức năng và có khả năng hoạt động ổn định trong phạm vi kiểm thử hiện tại, nhưng vẫn cần khắc phục các defect còn tồn tại và mở rộng phạm vi Automation cũng như Performance Testing trước khi có thể đánh giá đầy đủ chất lượng của hệ thống.

4.9. Kết luận chương

Chương 4 đã trình bày quá trình thực hiện và đánh giá kết quả kiểm thử đối với hệ thống FastFood Management System thông qua nhiều phương pháp kiểm thử khác nhau, bao gồm Manual Functional Testing, Unit Testing, Integration Testing, Automation Testing và Performance Testing.

Đối với Manual Functional Testing, các chức năng chính của hệ thống đã được kiểm tra thông qua các Test Case tương ứng. Kết quả cho thấy phần lớn các chức năng đáp ứng được Expected Result, đồng thời một số lỗi liên quan đến validation dữ liệu, xử lý lỗi và đồng bộ trạng thái giữa các module đã được phát hiện và ghi nhận.

Đối với Unit Testing và Integration Testing, kết quả JaCoCo cho thấy mã nguồn Backend đạt 92% Instruction Coverage, 72% Branch Coverage, 92% Line Coverage, 95% Method Coverage và 96% Class Coverage. Các chỉ số này cho thấy phần lớn mã nguồn đã được thực thi trong quá trình kiểm thử. Tuy nhiên, Branch Coverage vẫn còn thấp hơn các chỉ số còn lại và một số package cần được bổ sung Test để tăng mức độ bao phủ.

Đối với Automation Testing, Selenium WebDriver kết hợp với TestNG được sử dụng để tự động hóa các luồng Login, Add to Cart và Order. Bộ Test gồm 4 Test Case và cả 4 đều PASS, đạt tỷ lệ 100% trong lần thực thi được ghi nhận. Kết quả cho thấy Automation Testing có khả năng hỗ trợ hiệu quả cho việc kiểm tra các luồng nghiệp vụ lặp lại và Regression Testing.

Đối với Performance Testing, Apache JMeter được sử dụng để đánh giá khả năng xử lý của Login API và Orders API. Với 10 Concurrent Users và 5 Loop Count, tổng cộng 100 request được thực hiện. Kết quả đạt 0% Error Rate, thời gian phản hồi trung bình 13 ms, thời gian phản hồi lớn nhất 29 ms và Throughput đạt 97.8 requests/second. Điều này cho thấy hệ thống hoạt động ổn định trong phạm vi tải được kiểm thử.

Tổng hợp kết quả kiểm thử cho thấy hệ thống đã đáp ứng phần lớn các chức năng và luồng nghiệp vụ chính. Tuy nhiên, vẫn còn một số vấn đề cần tiếp tục cải thiện, đặc biệt là validation dữ liệu, xử lý lỗi API, các Test Case chưa đạt, các nhánh mã nguồn chưa được bao phủ và phạm vi Automation Testing còn hạn chế.

Nhìn chung, quá trình kiểm thử đã cung cấp cơ sở để đánh giá chất lượng của hệ thống trên nhiều khía cạnh khác nhau. Các kết quả thu được không chỉ giúp xác định mức độ đáp ứng của hệ thống mà còn chỉ ra những khu vực cần tiếp tục cải thiện nhằm nâng cao tính ổn định, khả năng bảo trì và chất lượng của FastFood Management System.

Chương 5. Đánh giá và Kết luận

5.1 Đánh giá kết quả đạt được

Sau quá trình xây dựng Test Plan, thiết kế Test Case và thực hiện kiểm thử cho hệ thống FastFood Management System, đề tài đã thực hiện kiểm thử trên nhiều mức độ khác nhau nhằm đánh giá tính đúng đắn, độ ổn định và chất lượng của hệ thống.

Các kết quả chính đạt được gồm:

- Xây dựng và thực hiện bộ Manual Functional Test Case cho các chức năng chính của hệ thống, bao gồm Login, Admin, Order and Cart, Kitchen, Payment and Table.
- Thực hiện kiểm thử Unit Testing và Integration Testing đối với Backend, đồng thời sử dụng JaCoCo để đánh giá mức độ bao phủ mã nguồn.
- Kết quả JaCoCo đạt 92% Instruction Coverage, 72% Branch Coverage, 92% Line Coverage, 95% Method Coverage và 96% Class Coverage.
- Xây dựng 04 kịch bản End-to-End Automation Testing bằng Selenium WebDriver và TestNG, bao phủ các chức năng Login, Add to Cart và Order.
- Kết quả Automation Testing đạt 4/4 Test Case PASS, tương ứng 100% Pass Rate trong lần thực thi được ghi nhận.
- Thực hiện Performance Testing bằng Apache JMeter đối với Login API và Orders API với 10 Concurrent Users và 5 Loop Count.
- Performance Testing thực hiện tổng cộng 100 requests, đạt 0% Error Rate, thời gian phản hồi trung bình 13 ms, thời gian phản hồi lớn nhất 29 ms và Throughput 97.8 requests/second.
- Ghi nhận và phân tích các lỗi phát hiện trong quá trình Manual Testing, tập trung vào validation dữ liệu, xử lý lỗi API và đồng bộ trạng thái giữa các module.
- Áp dụng nhiều công cụ hỗ trợ kiểm thử và đánh giá chất lượng mã nguồn như JUnit, Mockito, Spring Boot Test, Selenium, TestNG, JMeter và JaCoCo.

Việc kết hợp nhiều phương pháp kiểm thử giúp đánh giá hệ thống từ nhiều góc độ. Manual Testing tập trung vào hành vi và yêu cầu chức năng, Unit và Integration Testing kiểm tra mã nguồn và sự phối hợp giữa các thành phần, Automation Testing kiểm tra các luồng nghiệp vụ có tính lặp lại, trong khi Performance Testing đánh giá khả năng đáp ứng của hệ thống dưới tải.

Kết quả đạt được cho thấy hệ thống đã đáp ứng phần lớn các chức năng và luồng nghiệp vụ chính trong phạm vi kiểm thử. Đồng thời, quá trình kiểm thử cũng giúp xác định các khu vực còn tồn tại lỗi và cung cấp cơ sở để tiếp tục cải thiện chất lượng hệ thống.

5.2. Ưu điểm

5.2.1. Ưu điểm về quy trình kiểm thử

Đề tài đã xây dựng quy trình kiểm thử tương đối đầy đủ, bao gồm các hoạt động từ phân tích yêu cầu, lập Test Plan, thiết kế Test Case, thực hiện kiểm thử, ghi nhận kết quả và đánh giá chất lượng hệ thống.

Việc kết hợp nhiều phương pháp kiểm thử giúp quá trình đánh giá không chỉ tập trung vào chức năng mà còn xem xét mã nguồn, khả năng tích hợp, khả năng tự động hóa và hiệu năng của hệ thống.

5.2.2. Ưu điểm về phạm vi kiểm thử

Các chức năng chính của hệ thống đã được đưa vào phạm vi kiểm thử, bao gồm:

- Đăng nhập và phân quyền.
- Quản lý món ăn.
- Đặt món và giỏ hàng.
- Quản lý Bếp.
- Thanh toán.
- Quản lý bàn.

Việc xây dựng Test Case cho nhiều module giúp tăng khả năng phát hiện lỗi và đánh giá được tính nhất quán của hệ thống trong các luồng nghiệp vụ chính.

5.2.3. Ưu điểm về kiểm thử tự động

Đề tài đã áp dụng Selenium WebDriver và TestNG để tự động hóa các luồng nghiệp vụ chính.

Bộ Automation Test hiện tại gồm 4 Test Case và đạt 4/4 PASS, tương ứng 100% Pass Rate trong lần thực thi được ghi nhận.

Việc áp dụng Page Object Model và Data-driven Testing giúp mã nguồn Automation có tính tái sử dụng và thuận tiện hơn khi bảo trì, đồng thời tạo nền tảng để mở rộng Automation Test trong các lần phát triển tiếp theo.

5.2.4. Ưu điểm về đánh giá chất lượng mã nguồn

Việc sử dụng JaCoCo giúp đánh giá mức độ bao phủ mã nguồn Backend bằng các chỉ số Instruction, Branch, Line, Method và Class Coverage.

Kết quả đạt 92% Instruction Coverage, 92% Line Coverage, 95% Method Coverage và 96% Class Coverage, cho thấy phần lớn mã nguồn đã được thực thi trong quá trình kiểm thử.

Ngoài ra, việc sử dụng các công cụ như SonarQube/SonarCloud hỗ trợ đánh giá chất lượng mã nguồn và phát hiện các vấn đề cần cải thiện trong quá trình phát triển.

5.2.5. Ưu điểm về kiểm thử hiệu năng

Đề tài đã thực hiện Performance Testing bằng Apache JMeter nhằm đánh giá khả năng xử lý request của Backend.

Trong kịch bản kiểm thử với 10 Concurrent Users và 5 Loop Count, hệ thống xử lý 100 requests với 0% Error Rate, thời gian phản hồi trung bình 13 ms và Throughput đạt 97.8 requests/second.

Kết quả này cung cấp cơ sở để đánh giá khả năng đáp ứng của hệ thống trong điều kiện tải được thiết lập.

5.2.6. Ưu điểm về phát hiện và phân tích lỗi

Quá trình kiểm thử không chỉ xác định Test Case PASS/FAIL mà còn phân tích nguyên nhân của các lỗi được phát hiện.

Các lỗi chủ yếu được xác định liên quan đến:

- Validation dữ liệu đầu vào.
- Xử lý lỗi API.
- Đồng bộ trạng thái giữa các module.
- Các nhánh logic chưa được kiểm thử đầy đủ.

Việc phân tích nguyên nhân giúp kết quả kiểm thử có giá trị hơn trong quá trình cải thiện hệ thống, thay vì chỉ ghi nhận trạng thái PASS hoặc FAIL.

5.2.7. Đánh giá chung

Nhìn chung, ưu điểm nổi bật của đề tài là kết hợp nhiều phương pháp và công cụ kiểm thử thay vì chỉ thực hiện kiểm thử chức năng thủ công. Điều này giúp đánh giá hệ thống trên nhiều khía cạnh và tạo nền tảng cho việc tiếp tục phát triển quy trình kiểm thử tự động trong tương lai.

5.3. Hạn chế

5.3.1. Hạn chế về phạm vi kiểm thử

Mặc dù hệ thống đã được kiểm thử trên nhiều chức năng và nhiều mức kiểm thử khác nhau, phạm vi kiểm thử vẫn chưa bao phủ toàn bộ mọi tình huống có thể xảy ra trong thực tế.

Một số chức năng mới được kiểm tra ở mức chức năng thủ công hoặc chưa được tự động hóa đầy đủ. Đặc biệt, Automation Testing hiện tập trung vào Login, Add to Cart và Order, do đó chưa bao phủ toàn bộ các module của hệ thống.

5.3.2. Hạn chế về Automation Testing

Số lượng Automation Test hiện tại còn hạn chế với 04 Test Case. Các Test Case chủ yếu tập trung vào những luồng nghiệp vụ chính.

Các chức năng như Admin, Kitchen và Payment chưa được tự động hóa đầy đủ. Vì vậy, khi hệ thống có thay đổi, vẫn cần thực hiện nhiều Test Case thủ công để kiểm tra Regression Testing.

Ngoài ra, Automation Test phụ thuộc vào:

- Trạng thái của Frontend và Backend.
- Dữ liệu trong Database.
- Locator của các phần tử trên giao diện.
- Trình duyệt và môi trường thực thi.

Khi giao diện thay đổi, một số Selenium Test Script có thể cần được cập nhật.

5.3.3. Hạn chế về Code Coverage

Kết quả JaCoCo cho thấy Branch Coverage đạt 72%, thấp hơn Instruction Coverage, Line Coverage, Method Coverage và Class Coverage.

Điều này cho thấy vẫn còn một số nhánh điều kiện chưa được kiểm thử. Một số package cũng có mức Coverage thấp, đặc biệt là các phần liên quan đến Exception và Repository.

Do đó, cần bổ sung thêm Test Case cho các trường hợp ngoại lệ, dữ liệu không hợp lệ và các nhánh xử lý chưa được thực thi.

5.3.4. Hạn chế về Performance Testing

Performance Testing hiện mới được thực hiện với 10 Concurrent Users và 5 Loop Count, tổng cộng 100 requests.

Kết quả cho thấy hệ thống hoạt động ổn định trong cấu hình này, tuy nhiên chưa đủ cơ sở để xác định giới hạn chịu tải tối đa của hệ thống.

Chưa thực hiện đầy đủ các mức kiểm thử như:

- Stress Testing với số lượng người dùng lớn.
- Load Testing trong thời gian dài.
- Spike Testing với tải tăng đột biến.
- Soak/Endurance Testing trong thời gian kéo dài.

Do đó, khả năng hoạt động của hệ thống khi số lượng người dùng tăng cao vẫn cần được đánh giá thêm.

5.3.5. Hạn chế về môi trường kiểm thử

Các kết quả kiểm thử chủ yếu được thực hiện trong môi trường Local. Vì vậy, kết quả có thể chưa phản ánh hoàn toàn hiệu năng và hành vi của hệ thống trong môi trường triển khai thực tế.

Ngoài ra, việc kiểm thử chưa được thực hiện đầy đủ trên nhiều trình duyệt, thiết bị và cấu hình môi trường khác nhau.

5.3.6. Hạn chế trong việc quản lý Defect

Các defect được phát hiện trong quá trình kiểm thử đã được ghi nhận và phân tích, tuy nhiên quy trình quản lý defect chưa được triển khai đầy đủ trên một hệ thống chuyên biệt.

Việc áp dụng các công cụ quản lý như Jira hoặc một hệ thống Defect Tracking chuyên dụng trong các giai đoạn tiếp theo sẽ giúp theo dõi trạng thái, mức độ ưu tiên, người xử lý và lịch sử của từng defect hiệu quả hơn.

5.3.7. Đánh giá chung về hạn chế

Những hạn chế trên chủ yếu xuất phát từ phạm vi và thời gian thực hiện đề tài. Tuy nhiên, các kết quả hiện tại vẫn cung cấp cơ sở để đánh giá chất lượng hệ thống và xác định những hướng cần tiếp tục cải thiện.

Các hạn chế này cũng là cơ sở để đề xuất hướng phát triển và mở rộng hoạt động kiểm thử trong phần tiếp theo.

5.4. Hướng phát triển

5.4.1. Mở rộng phạm vi Automation Testing

Trong tương lai, hệ thống có thể tiếp tục mở rộng Automation Testing từ 4 Test Case hiện tại sang các chức năng còn lại như Admin, Kitchen, Payment và Table.

Bên cạnh các luồng chức năng chính, có thể bổ sung thêm các trường hợp:

- Dữ liệu hợp lệ và không hợp lệ.
- Boundary Value Testing.
- Các trường hợp Exception.

- Các luồng nghiệp vụ có nhiều điều kiện.
- Regression Test sau mỗi lần cập nhật hệ thống.

Việc mở rộng Automation Test sẽ giúp giảm khối lượng kiểm thử thủ công và tăng khả năng phát hiện lỗi sau mỗi lần thay đổi mã nguồn.

5.4.2. Cải thiện Code Coverage

Kết quả hiện tại cho thấy Branch Coverage đạt 72%, do đó cần tiếp tục bổ sung Test cho các nhánh điều kiện chưa được thực thi.

Các Test bổ sung nên tập trung vào:

- Các nhánh if/else.
- Các trường hợp Exception.
- Dữ liệu đầu vào không hợp lệ.
- Các trường hợp Database không trả về dữ liệu.
- Các trường hợp API trả về lỗi.
- Các trường hợp nghiệp vụ đặc biệt.

Mục tiêu là nâng cao Branch Coverage đồng thời duy trì mức độ bao phủ cao đối với Instruction, Line, Method và Class.

5.4.3. Mở rộng Performance Testing

Performance Testing có thể được mở rộng với nhiều mức tải khác nhau thay vì chỉ sử dụng 10 Concurrent Users.

Các kịch bản có thể thực hiện trong tương lai gồm:

- Load Testing: tăng dần số lượng người dùng để xác định khả năng xử lý của hệ thống.
- Stress Testing: tăng tải vượt mức dự kiến để xác định giới hạn hệ thống.
- Spike Testing: mô phỏng trường hợp số lượng request tăng đột biến.
- Endurance Testing: duy trì tải trong thời gian dài để phát hiện vấn đề về tài nguyên hoặc hiệu năng.

Ngoài ra, có thể theo dõi thêm CPU, RAM, Database Connection Pool và các tài nguyên hệ thống để xác định chính xác nguyên nhân khi hiệu năng giảm.

5.4.4. Mở rộng môi trường kiểm thử

Các lần kiểm thử tiếp theo có thể được thực hiện trên nhiều môi trường và trình duyệt khác nhau như Chrome, Edge và Firefox.

Việc kiểm thử trên nhiều môi trường giúp phát hiện các vấn đề liên quan đến:

- Khả năng tương thích trình duyệt.
- Hiển thị giao diện.
- JavaScript và CSS.
- Khác biệt về tốc độ xử lý.
- Khác biệt giữa môi trường Local và môi trường triển khai thực tế.

5.4.5. Cải thiện quản lý Defect

Có thể tích hợp một công cụ quản lý Defect chuyên dụng như Jira vào quy trình kiểm thử. Mỗi defect có thể được quản lý theo các thông tin:

- Defect ID.
- Summary.
- Description.
- Severity.
- Priority.
- Steps to Reproduce.
- Expected Result.
- Actual Result.
- Assignee.
- Status.
- Resolution.

Điều này giúp quá trình theo dõi và xử lý defect có hệ thống hơn, đồng thời thuận tiện cho việc thống kê và lập báo cáo chất lượng phần mềm.

5.4.6. Tích hợp Automation Testing vào CI/CD

Automation Test có thể được tích hợp vào quy trình CI/CD để tự động thực thi sau mỗi lần thay đổi mã nguồn.

Quy trình có thể được mở rộng theo hướng:

Developer Commit → Build → Unit Test → Integration Test → Automation Test → Report → Deploy

Khi một Test Case thất bại, Pipeline có thể thông báo để nhóm phát triển kiểm tra và xử lý trước khi triển khai phiên bản mới.

Việc tích hợp CI/CD sẽ giúp hoạt động kiểm thử được thực hiện thường xuyên hơn và hỗ trợ phát hiện lỗi sớm trong vòng đời phát triển phần mềm.

5.4.7. Hướng phát triển tổng thể

Trong tương lai, hệ thống kiểm thử có thể được phát triển theo hướng tự động hóa và tích hợp toàn diện hơn, bao gồm Unit Testing, Integration Testing, End-to-End Testing, Performance Testing và CI/CD.

Mục tiêu là xây dựng một quy trình kiểm thử có khả năng thực hiện tự động, tạo báo cáo kết quả và phát hiện lỗi sớm sau mỗi lần thay đổi hệ thống. Điều này sẽ góp phần nâng cao chất lượng, độ ổn định và khả năng bảo trì của FastFood Management System.

5.5. Kết luận

Sau quá trình nghiên cứu, xây dựng kế hoạch và thực hiện kiểm thử cho hệ thống FastFood Management System, đề tài đã áp dụng nhiều phương pháp kiểm thử khác nhau nhằm đánh giá tính đúng đắn, độ ổn định và khả năng đáp ứng của hệ thống.

Quá trình kiểm thử bao gồm Manual Functional Testing, Unit Testing, Integration Testing, End-to-End Automation Testing và Performance Testing. Bên cạnh đó, các công cụ như JUnit, Mockito, Spring Boot Test, Selenium WebDriver, TestNG, Apache JMeter và JaCoCo được sử dụng để hỗ trợ quá trình kiểm thử và đánh giá chất lượng phần mềm. Việc sử dụng nhiều mức kiểm thử phù hợp với mục tiêu của Test Plan là đánh giá chức năng, sự tương tác giữa Frontend, Backend và Database cũng như khả năng hoạt động khi có nhiều người dùng truy cập.

Kết quả kiểm thử cho thấy các chức năng chính của hệ thống đã được kiểm tra và phần lớn đáp ứng Expected Result. Automation Testing đạt 4/4 Test Case PASS, trong khi Performance Testing với 10 Concurrent Users và 5 Loop Count ghi nhận 100 requests, Error Rate 0%, Average Response Time 13 ms và Throughput 97.8 requests/second.

Đối với mã nguồn Backend, JaCoCo ghi nhận mức 92% Instruction Coverage, 72% Branch Coverage, 92% Line Coverage, 95% Method Coverage và 96% Class Coverage. Các kết quả này cho thấy phần lớn mã nguồn đã được thực thi trong quá trình kiểm thử, đồng thời vẫn còn một số nhánh và thành phần cần được bổ sung Test.

Bên cạnh những kết quả đạt được, quá trình kiểm thử cũng phát hiện một số hạn chế liên quan đến validation dữ liệu, xử lý lỗi API, đồng bộ trạng thái giữa các module và phạm vi Automation Testing. Đây là những cơ sở quan trọng để tiếp tục cải thiện hệ thống.

Trong tương lai, hoạt động kiểm thử có thể được mở rộng bằng cách tăng phạm vi Selenium Automation, bổ sung Unit và Integration Test, thực hiện Performance Testing với mức tải cao hơn, mở rộng kiểm thử trên nhiều trình duyệt và tích hợp toàn bộ quy trình kiểm thử vào CI/CD. Các hướng này cũng phù hợp với định hướng phát triển được đề xuất trong tài liệu dự án.